Everything but the Software:

Feature-Driven Decay in an Agentic Coding Loop. An Exploratory Case Study and an Experiment Protocol

Juan Manuel Servera Bondroit*

Senior Partner Solution Architect, Microsoft Schweiz, Zurich

Draft of 1 November 2026

Abstract

Each feature changes the starting point for the next. In agentic software development, that starting point includes code, tests and the prompts, skills and notes directing subsequent work. This paper examines how faults and corrective instructions can accumulate across this process. Industrial maintenance accounts, recent agent benchmarks and public project records motivate a distinction between errors within a conversation and errors inherited through a repository. A paired-history benchmark supplies evidence of the latter, while its clean-history results qualify an explanation based on structural complexity alone. A six-week architectural demonstrator provides a motivating testbed: 183 pull requests merged from 15 to 24 October 2026, 16:19:39 UTC, while the replacement remained unusable for the participant. Five candidate mechanisms organise its delivery record: specification inflation, proxy capture, learning evaporation, process-gate inflation and feedback starvation. The case centres on a migration with missing deployment dependencies; it exposes a break in usable behaviour rather than measuring a decay rate. A prospective protocol specifies 495 trajectories across three selected models and five arms: feature-only delivery, scheduled consolidation, read-only review, matched-resource feature delivery and visible regression repair. Its principal outcomes are final behavioural coverage, retained-contract loss, an unforeseen change request and resource cost. The consolidation arm includes additional resources, tests and a different handover note. The sample targets a ten-percentage-point coverage gain under an explicit variance assumption. Affordability, a new source freeze, prospective OSF recording and collection authorization remain gates. No experiment results are reported. The study asks whether consolidation improves the system inherited by later feature work.

Keywords: agentic software engineering; coding agents; spec-driven development; software evolution; technical debt; refactoring; feedback loops; human–AI interaction; exploratory case study; experiment protocol.

1 Introduction

On 1 August 2012, Knight Capital's automated router sent more than four million orders while attempting to fill 212 customer orders. The firm lost more than USD 460 million. The SEC traced the incident to a defective function left in the router after a 2005 change, an incorrect deployment for a new trading programme, and inadequate controls. The new deployment activated old code that was no longer meant to run [96]. The failure crossed a feature boundary and a seven-year interval: the relevant starting point was the accumulated system, not just the latest patch.

*The views expressed are the author's own and do not necessarily reflect those of Microsoft. The case organisation, Zava North Homes, is fictional.

Less spectacular versions of the same maintenance problem are familiar. Cunningham's 1992 account of WyCash described a financial product continually adapted to market demands. Its mature code was repeatedly revised and rewritten; the technical-debt metaphor explained why shipping a first-time implementation required later rewriting as understanding improved [29]. Some debt is visible as duplication or awkward dependencies. Some remains hidden in assumptions about inputs, deployment and earlier behaviour until another change depends on it. Counting completed changes reveals neither.

Agentic development makes this distinction newly pressing. PhotoCraft, an early-alpha reimplementation of Photoshop, reports 631 of 631 menu items live, but explicitly defines that count as dispatch rather than behaviour. Its own dated assessment estimates about 76% feature breadth and 45% readiness for real work [84]. A command can be reachable without the surrounding workflow being dependable. At the repository level, SWE-Maintain finds that agents solve fewer later tasks when starting from their own earlier changes than from the developers' changes, even though each task starts in a fresh session [78]. The next session can forget the conversation and still inherit its mistakes.

The engineering response has a long history: isolation and explicit contracts limit dependencies; tests and continuous integration expose regressions; small releases shorten feedback; refactoring preserves room for later change [8, 9, 36, 80]. Recent discussions by Beck, Martin and Fowler revisit these concerns for agent-assisted work [16, 37, 66]. Beck's distinction between producing code and learning through working software sharpens the research question [14, 16]: what does a feature leave behind for the next one?

The question arose during an Azure Master Architect (AMA) capstone, used here as an exploratory testbed rather than the scope of the contribution. After intermittent use of an MVP, its sole developer started a coordinator on 15 October 2026 to deliver more than 180 backlog issues. From 15 October through the 24 October integration at 16:19:39 UTC, 183 pull requests merged, but the developer could not sign in to the replacement. The public repository, pull requests, append-only log and versioned coordinator instructions provide a trace of that transition [89]. The case motivates hypotheses, not a general estimate of agent-caused deterioration. The author's earlier experience with Aithena, a book-search project with PDF ingestion and a browser interface discussed separately in the supplement, and the PhotoCraft inspection selected after the AMA analysis have separate provenance in the ancillary anc/supplementary-context.md.

The working hypothesis is that feature-driven delivery can carry unresolved errors and avoidable change burden forward unless it includes cumulative verification and consolidation. Two kinds of state matter: the software being changed, and the instructions used to change it. Adding a corrective rule after every failure can leave prompts, skills and handover notes with their own maintenance burden. A fresh agent inherits both kinds of state. Section 2 distinguishes these pathways; the case then examines how they interact with completion proxies, review capacity and product-use feedback.

Here *feature-driven decay* describes loss of retained behaviour or increasing change burden across feature steps. Whether feature-driven decay occurs remains a question for the prospective study.

Three questions guide the investigation:

- **RQ1.** Through which retained state can errors and change burden carry from one feature to the next, and what does existing evidence show about those pathways?
- **RQ2.** How does an exploratory delivery record illuminate that problem, and which candidate mechanisms and rival explanations does the gap between completion signals and usable replacement behaviour suggest?
- **RQ3.** Does scheduled agent-led consolidation after each feature improve retained behaviour and future changeability, and at what cost?

The report makes three linked contributions:

1. A focused synthesis distinguishing conversational self-conditioning, repository-carried faults and instruction maintenance, with a replot of published paired-history data (Section 2).
2. A documented six-week case, including a ten-day autonomous delivery loop, and an integrated account of five mechanisms. Descriptive measures, within-case contrasts and an evidence audit connect the observed interruption of usability to competing explanations (Sections 3–5).
3. A consolidation harness and a prospectively sized five-arm protocol to estimate retained behaviour, future changeability and resource cost. It follows the registered-report example of Lulla et al. [63] and HVE Core's Minimum Viable Experiment method (Sections 6 and 7).

The case concerns a migration and a configuration designed by the author, including delegated delivery and merge decisions. The case suggests hypotheses whose causal relationships remain to be tested. The planned pilot estimates a multi-component, extra-resource package comparison; it does not isolate refactoring or validate the five case mechanisms individually. It is not running, and registration and launch checks remain pending. The case's validity and the pilot's identification limits are consolidated in Sections 5.2 and 7.3.

2 Background

2.1 Accumulation and inherited state

Lehman's laws and Parnas's software-aging account motivate maintenance alongside growth [58, 59, 80]. The 1997 formulation states the declining-quality law; it supplies no data for or against it. Eick et al. report mixed evidence from fifteen-plus years of change history [34], and a systematic review finds uneven support across systems [46]. Neither feature count nor churn is itself decay. A regression loses still-required, previously working behaviour; an unimplemented capability is an acquisition failure. Repeated failure is persistence, not proof that an earlier defect caused a distinct later one.

Cunningham describes shipping first-time code and rewriting it as understanding improves [29]. Fowler's design stamina is explicitly a hypothesis [36]. Reported debt costs concern different settings: a vendor-associated 39-codebase study reports 124% longer issue resolution in low-quality files [95]; 43 developers self-report about 23% of working time lost to debt [18]. These are not universal productivity coefficients.

What survives the next session? Code, tests and deployment assumptions survive through the repository; instructions and handovers survive through procedural context. Either channel can preserve a sound decision or a mistake. Here "software entropy" is an engineering analogy, not a thermodynamic measure. The proposed outcomes are retained behaviour, adaptation and resource use.

2.2 Evidence within and between features

Conversational benchmarks identify dependence on erroneous histories, not merely context length. Zhang et al. study false justifications after wrong answers [103]; Sinha et al. vary erroneous prior state updates in a controlled execution task [92, Secs. 2.2–3, Figs. 5–6]. Laban et al.'s 39% relative multi-turn performance drop concerns simulated underspecified conversations, not repository defects; locators refer to their arXiv v1 [56, Secs. 3.3, 6.1, App. F].

Pan et al.'s SWE-Maintain more directly tests repository inheritance: 100 three-event episodes from 50 Python repositories, with fresh sessions and matched later tasks under developer and agent histories. Across 1,996 paired later events, agent history reduces current-task

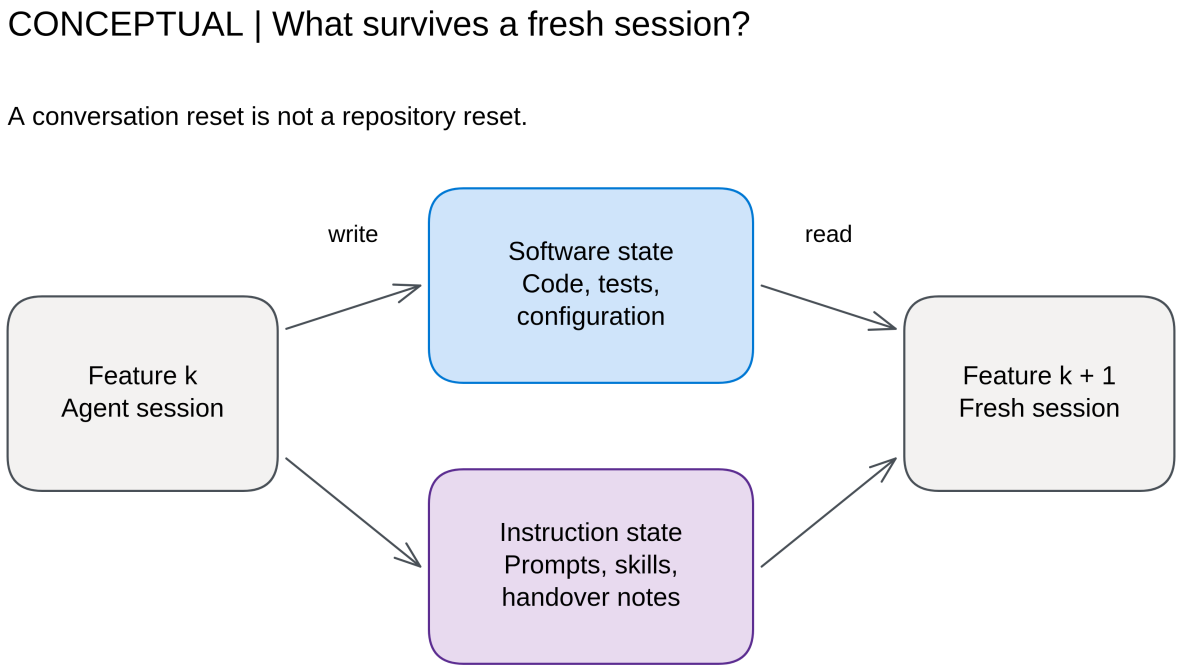


Figure 1: Conceptual read/write paths between sessions. A fresh conversation does not reset software or instructions. Arrows are possible state transfer, not measured error propagation.

resolution by 5.6 points; requiring earlier tests also to pass gives a 14.0-point gap. Unweighted mean current-task gaps grow from 4.0 to 7.2 points between the second and third events. In the selected 619-event clean-history subset, the current-task difference is -1.1 points, interval $[-3.8, +1.6]$, and cumulative event resolution differs by $+2.7$, interval $[-0.5, +6.1]$ [78, Secs. 4.1–4.2, Tables 2–3]. This favours inherited behavioural failures over complexity alone as the main observed channel, without establishing a general decay law or a randomised repair effect.

Their `python_unidiff` example reuses line-splitting logic that loses terminators in a later constructor; Figure 1's caption introduces it and Section 5.1/Figure 5 supplies the detail [78]. SlopCodeBench and SWE-CI extend iterative evaluation [26, 77]. Borg et al.'s maintainability study finds no significant downstream difference for late-2024 assistants; only phase two was randomised [21]. This is a counterexample, not a test of the current autonomous roster.

2.3 Instruction maintenance and evaluation

Instructions are maintained artifacts, not automatically reliable memory [76, 85]. Mohsenimofidi et al.'s 169 commits have 242 overlapping assignments across nine categories; 78 instruction additions, 59 modifications and 23 removals are three categories, not a partition [71, Sec. 4.3, Table 2]. Chatlatanagulchai et al.'s sampled maintenance labels likewise include addition and pruning [24]. Instruction debt here is the author's term for obsolete, duplicated, conflicting or poorly routed directives.

Gloaguen et al. report 20%/23% higher inference cost with LLM-written context and slightly lower success, by 0.5/2 points, without significant differences from no-file baselines. Developer-written files outperform LLM-written ones on CTXbench ($p = 0.038$) [41, Sec. 4.2, Tables 2–3]. SkillOpt and Jiang et al. provide counterevidence to treating retained instructions as inherently harmful: evaluated procedural guidance can help, while retrieval and downstream success remain

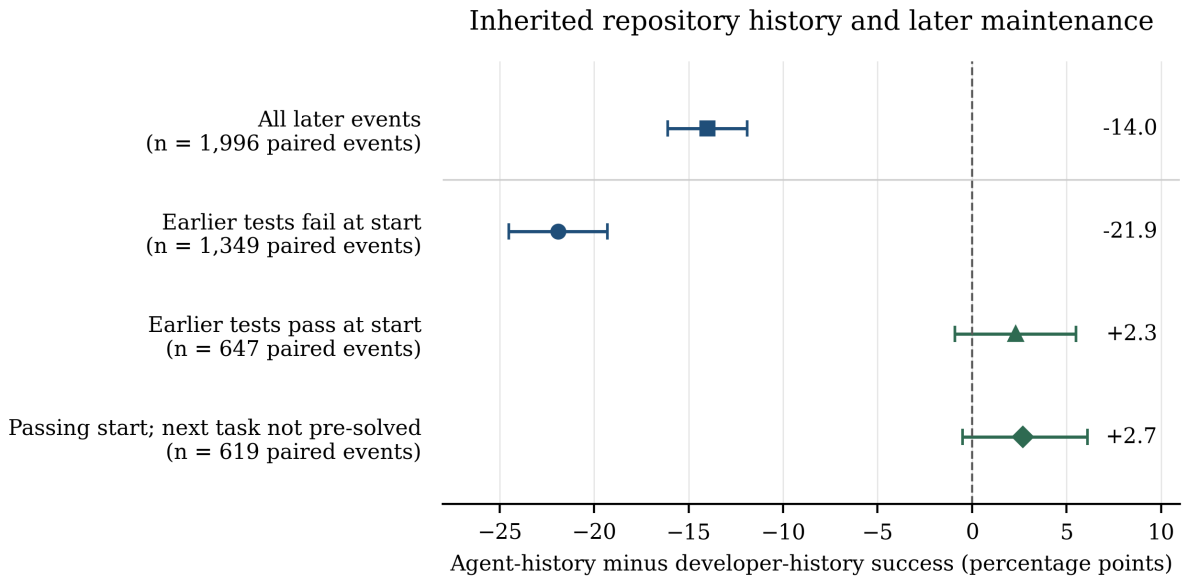


Figure 2: Published event-resolution differences and repository-clustered 95% intervals from Pan et al., Table 3. Current and prior non-superseded tests must pass. Selected cohorts overlap; they are not independent studies or randomised repair conditions. Values and whiskers are transcribed, not fitted here.

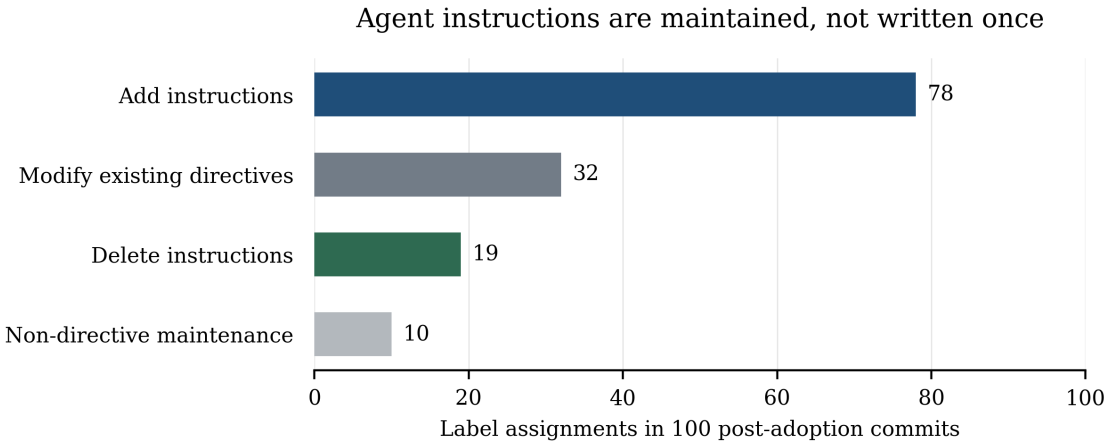


Figure 3: Labels in 100 sampled post-adoption commits, from Chatlatanagulchai et al., Sections 4.2.2–4.2.3. Labels overlap and count edit types, not net token growth or effects on agent success.

distinct [51, 70].

Tests and review. RefactorBench studies refactoring, not between-feature consolidation [40]; developer-agent collaboration studies are observational [55]. SWE-Sharp-Bench supplies reproducible repository evaluation [67], while CRBugBench exposes review limits [100]. Building to the Test finds a dead or absent required library in seven of twelve oracle-in-the-loop runs [64]. ImpossibleBench tests exploitation more broadly [104]. Test integrity and adequate outcome coverage are different requirements. AgentRx motivates traceable failure diagnosis [68].

Bounded correction. Pavlenko et al.'s factory optimises human-architected systems against measurable objectives; productivity estimates cover application work, excluding infrastructure and onboarding [81]. AutoSaddler treats harness updates as offline learning [79]. Neither

isolates the present intervention and later change probe.

Reflection and self-improvement. Ng's reflection design-pattern letter explicitly recommends Self-Refine, Reflexion and CRITIC [75]; he is not a coauthor of those papers. Self-Refine iterates feedback and revision with the same model without training [65, Sec. 2]. Reflexion retains linguistic feedback in episodic memory for subsequent trials rather than updating model weights [90, Sec. 3]. CRITIC uses external tools to check and revise an output [44, Sec. 3]. These are precedents for reflective correction, not evidence that every retained lesson or revision improves later feature work. Their feedback and repeated-trial budgets differ from the present protocol, whose hidden evaluator outputs never reach agents.

Ng coauthored MedAgentBench, which evaluates task completion in an interactive medical-record environment rather than chatbot answers alone [50, arXiv v2, Secs. 2.3–2.4]. It motivates checking observable action outcomes, not a claim about coding consolidation or a controlled self-improvement intervention.

The two owner-selected videos illustrate these boundaries rather than provide experimental findings. In the Ng/Moroney talk, Ng describes human-feedback iteration; Moroney later warns about code structure and describes task-level reflection [83]. In the separate memory presentation, a speaker identifying herself as Anthropic staff describes persistent memory and asynchronous transcript-driven curation, with human review of proposed updates [6]. Both sources are third-party uploads checked through automatic captions, not audio-verified research reports. Neither supplies a controlled cross-feature consolidation result.

Engineering rationale. Beck's alternation of features and tidying aims to restore future options [11–13, 16]. Extending that metaphor to acceptance criteria is this author's interpretation. Understanding/abstraction and learning through use have separate sources [15, 16]. Tests, short releases, explicit design responsibilities and usable increments motivate the checklist [7–9, 22, 27, 38, 39, 49, 66, 74, 87]. Monitoring burden and completion proxies qualify the rationale [3, 28, 43, 57, 91, 93, 94]. Spec-driven tools add structure, not an outcome oracle [1, 20, 30, 97].

HVE Core. Microsoft's Hypervelocity Engineering Core supplies file-based Research–Plan–Implement–Review handovers; the author's coordinator added autonomous dispatch. The reported case snapshot is `plugins-v3.3.106`, not proof of its installation date [69]. Full literature and framework detail is in the supplement.

3 Case and method

3.1 Setting

The motivating testbed is a capstone for Microsoft's *Azure Master Architect* (AMA) programme. The programme's name is public in the Microsoft Employee Learning badge listing.[1] That public listing does not authorise publication of internal assessment material or this paper. Case Study 48 supplies a fictional not-for-profit housing association, Zava North Homes, with about 60,000 homes. Its retrofit demonstrator combines assessment, grants, procurement, resident consent and quality assurance. The exercise required research, requirements, architecture, a delivery plan, a working demonstrator and an executive presentation. Domain obligations provide realistic dependencies; no actual housing service or resident data is evaluated here. The assessment concerned implementation completeness. Internal rubric wording is not reproduced; its external-release clearance has not been established.

The author worked alone, as architect and product owner, from 11 September to 24 October 2026. Agents did almost all of the research synthesis, writing and coding. The repository also

[1] https://www.credly.com/org/microsoft-employee-learning/badge/azure-master-architect

contains an automated scorer: a GitHub agentic workflow that asks a language model to score each pull request against the rubric and records the result [89].

3.2 Workflow and tools

Interactive development used GitHub Copilot in VS Code, mainly Opus 5.5 during the first two weeks and then GPT-6.1 Sol through the case cut-off. These are recalled interface labels, not verified provider build identifiers or an exact switch date.

- **Research.** Sixteen official legislation, regulator, policy, standards and funding snapshots were downloaded on 19 September 2026, converted to Markdown and organised, with agent-written sector and company research, into a knowledge base: an index, immutable sources, maintained syntheses, an owner-curated memory file and an append-only log. Snapshot dates do not establish currentness. The 29 October paper and presentation claim review records the source dates and unresolved rechecks; no official pages could be retrieved in that session.
- **Challenge.** Eight agents role-playing the case organisation's stakeholders (asset management, programme delivery, finance, procurement, data, building safety, neighbourhood housing and resident engagement) and HVE Core's Design Thinking coach challenged each idea. Their first verdict was that none of the five end-to-end agents requested in the brief should be built as such.
- **Requirements and decisions.** HVE Core's BRD and PRD builders, a BRD quality reviewer and the ADR assistant produced and revised the requirements and six architecture decision records.
- **Prototypes and MVP.** Several prototypes were built, reviewed by the role agents and kept as evidence. An MVP followed and was deployed to Azure.
- **Backlog.** HVE Core's backlog manager turned the requirements and decisions into GitHub issues with epics, features, acceptance criteria and dependencies, labelled by how much of each item already existed in the MVP.
- **Coordinator.** On 15 October the author added a custom coordinator to deliver the backlog (Section 3.3).

3.3 The coordinator

The coordinator ran through Copilot CLI on a VM. It selected waves of issues whose dependencies were merged and launched parallel worker CLI sessions through `tmux`, each in an isolated git worktree with HVE Core's RPI agent and one issue as its brief. The author reports using Sonnet 5.5 for the workers, including before the explicit pin. At the pinned repository commit the worker selector is `claude-sonnet-5.5`; PR #420 introduced that requirement on 21 October. Earlier use is recollection; the committed selector independently records the later configuration. The coordinator used Auto selection with the Intelligence setting. The author observed it tending to select GPT-6.1 Sol, but this was not a guaranteed model pin or proof of every session's actual model or provider build. Within its first day the author added four rules, each in response to a reported problem. First, it could merge its own pull requests through a gate it verified itself, because repository auto-merge was unavailable. Second, the full research, plan, implement and review loop became mandatory for every issue, and the presence of the loop's `.copilot-tracking` artefacts in the pull request became a merge condition, because the first waves of sessions had implemented directly without them. Third, it updated the project board and ticked only the acceptance criteria that reviewed evidence proved. Fourth, it monitored sessions on a schedule instead of blocking. Later rules required a complete local validation gate before every push, including documentation-only changes, covering five source projects, infrastructure templates, a disposable SQL Server, browser accessibility checks, migrations, four

container images, vulnerability scans and a software bill of materials, to be rerun after every correction or rebase. An issue stopped only on a *human blocker*: an owner decision, a named approval, a missing credential, or a review finding still open after two correction cycles. On the same day an accepted ADR made migrating the working MVP into a new source layout the first priority and barred new feature code from the MVP.

3.4 Data and analysis

The study uses an exploratory single-case design, with the author as participant-observer. The case is the delivery of this repository from 11 September to 24 October 2026. The unit of analysis is the autonomous loop of 15–24 October. The loop is compared with the preceding MVP period of 8–14 October within the same case, acknowledging the uncontrolled differences between them. The author used the system intermittently during that period to refine ideas and the user experience. The case was selected because it is revelatory: a contemporaneous trace of an autonomous agent loop. Selection occurred after the outcome was known, not through prospective sampling of successful and unsuccessful projects.

The analysis uses the public repository at commit `29c6b51` (24 October 2026, 16:19 UTC) [89] with issue and pull-request metadata retrieved from the GitHub API on the same day. The main sources are the append-only knowledge-base log (368 dated entries and about 57,000 words from 11 September to 22 October; entries before 25 September were reconstructed from history), the knowledge-base index, the BRD (version 0.19), the PRD (version 0.18), the coordinator's definition, issue bodies and the rubric scorer's latest result. Methods comprise descriptive counts; an evidence-boundary lexicon applied to each log entry (Appendix A.6); reference tracing of recorded learning identifiers through the log by date; and AI-assisted interpretation of the author's observations. These informed the candidate mechanisms, possible amplifiers and consolidation hypothesis. GitHub Copilot in VS Code organised those explanations, named the five patterns and drafted the account, using Opus 5.5 and then GPT Sol 6 and 6.1 as reported interface labels. Microsoft 365 Copilot helped summarise material and find relevant reference documents; it did not perform the log analysis or develop the mechanisms. This was not a systematic entry-by-entry qualitative coding study. The author reviewed important specification sections selectively, especially the addendum, rather than auditing the full BRD and PRD. Their identifiers and cross-references made the overall design difficult to follow. The author reports checking the repository figures and Azure expenditure. The supplied AI-assisted critique reports checking the cited numbers and log-growth chart. This revision independently reruns selected local arithmetic and source comparisons, with coverage recorded in the package README. Neither check is independent human peer review or an audit of the original studies' data. The central discrepancy needs a compact, release-linked evidence package: the dated score and its assessed capability scope, the deployment revision and configuration, and dated evidence of the failed sign-in journey. The retained account does not yet establish that the scorer assessed the same release and capability claim as the usability observation. Until those links are checked, their coexistence is reported in the case chronology, not a verified same-release contradiction. The report is informed by established guidance for case study research in software engineering [88] and the ACM SIGSOFT Empirical Standards [86]. As the sole developer, the author is a participant-observer; Section 5.2 discusses what that implies.

The contextual observations in the ancillary supplement were added on 25 October after the case analysis. They use purposively selected public repositories, pinned source files, GitHub tracker counts, and ten newest plus five oldest open `bug`-labelled PhotoCraft reports. They are not a systematically sampled multi-case study and are not pooled with AMA's measurements. The ancillary repository-observation record states the retrieval boundary, locators and exclusions. No external application, original Photoshop installation or reference-parity suite was executed.

The literature-led framing was developed on 27 October. Source selection followed three questions: how earlier defects affect later changes, how agent instructions evolve, and how

completion measures relate to usable behaviour. Sources comprise the SEC's Knight Capital account, the original WyCash report, the dated public repository records and the cited empirical studies. Knight Capital provides a documented inherited-fault example; PhotoCraft provides a project-authored distinction between command coverage and readiness. Their outcomes are not pooled with AMA. The two new figures replot published values rather than digitising curves or running the original systems. Versioned claims, locators and transcription details are retained in the source package README and anc/published-evidence.json. The review offers a focused narrative synthesis, with no claim to systematic coverage. The two sources added on 29 October were selected by the owner for relevance to judgment and skill reuse, not through an updated systematic search.

3.5 Constructs and evidential boundaries

Here an *agent* is a configured model–tool–instruction loop that selects and executes actions, not the model alone or an independently accountable person. A *skill* is a reusable procedural resource describing when and how to act and verify the outcome, distinct from an execution history or a model-weight update. This procedural distinction follows Jiang et al. [51, Sec. 1]; the pilot's handover note is retained context, not a separately manipulated skill treatment.

Working behaviour means completing an intended task through the running system, including the identity, data and service dependencies needed for that task. A healthy endpoint or a passing repository test alone does not establish it. *Decay* denotes loss of previously working behaviour or increasing difficulty of understanding and changing a system. The case records lost usability during replacement; any decline in intrinsic code quality remains unmeasured. Deployment drift could produce the same observation. *Adaptability* is operationalised prospectively as success and cost on the fixed-budget, unforeseen change request, not as a universal measure of maintainability. *Consolidation* means scheduled structural work intended to preserve observable behaviour, with tests checking that intention; passing finite tests cannot prove semantic equivalence.

Five measurement categories remain separate. PRs, lines and words describe *production volume*; checks, reviews and receipts describe *validation activity*; observed user tasks describe *system behaviour*; code metrics and the change request probe *internal quality and changeability*; and corrective help requested or supplied describes *intervention*. The original observations cover the first three. November's post-hoc comparison adds structural proxies, not a controlled changeability task or systematically coded intervention dataset. Neither volume nor validation activity is used as a direct estimate of decay.

3.6 Chain of evidence and reflexivity

Table 1 separates observations, interpretations and propositions. The table organises the evidence already presented; it adds no newly coded observations. Repository sources refer to the pinned commit unless identified as author reports. Log entries before 25 September are retrospective reconstructions; later dated entries are the project's contemporaneous record, not independent corroboration of every statement they contain. The 22 and 24 October instructions, historical model use and intermittent product use remain author reports where original records have not been recovered.

The author selected the episodes, designed the coordinator and had an interest in successful delivery. Those roles make a coherent retrospective explanation attractive. Developers' experience-based beliefs need not correspond with project evidence [31]; that methodological caution applies to the author's account as well as agent-generated claims. AI-written records are treated as accounts of project activity, with AI also assisting synthesis. Neither provides independent corroboration. No independent second coding or empirical peer-debrief pass has been completed. The supplied editorial critique prompted stronger boundaries and explicit rival explanations, but does not fill that gap. An independent check of O1–O5 against retained source

Table 1: Observation–interpretation–proposition audit trail. Interpretations are contestable; the proposed tests have not been performed.

ID	Observation and provenance	Interpretation	Proposition or discriminating evidence
O1	Log, 8–14 Oct, plus author report: an MVP with sign-in and realistic synthetic data was used; 16 Oct record: replacement has one case, no staff and no model.	Recorded engineering completion did not preserve the owner's usable workflow.	Independently execute the same journey before and after migration; separate configuration failure from code regression.
O2	GitHub metadata: 183 PRs merged, 15–24 Oct; scorer records 5/5 completeness; author could not sign in. Dated API/scorer captures and their release/scope correspondence remain unverified.	Recorded completion and usability differed in the chronology; a same-release discrepancy is not yet established.	Link score, capability scope, deployment/configuration and the failed journey before testing agreement.
O3	BRD/PRD versions and log: growth before and during the loop; PR #445: two production lines added, one removed, and a 78-line evidence record.	Requirements and process records may consume attention without delivering equivalent feedback.	Compare review effort and validated behaviour, not word counts alone.
O4	Log reference tracing: last simplicity citation on 13 Oct; missing registration/staff bindings in the replacement.	Some lessons may have fallen off the effective read path. Absence of a citation is not proof of absence of knowledge.	Inspect actual session inputs and test whether the prior lesson was available, understood and executable.
O5	Coordinator definition and ADR 0006: committed RPI records, full gates, delegated merging and migration priority; author still resolved blockers.	Reduced product-use feedback coexisted with substantial oversight.	Distinguish outcome-level feedback from approval activity; test the consolidation package separately from the migration.

and execution records remains necessary before presenting the interpretations as independently corroborated. Jiang et al. make mechanism analysis inspectable through matched trajectories, evidence-linked labels and a separate human check of taxonomy construction, while distinguishing that check from exhaustive human coding [51, Sec. 4, App. A.2]. Here the five mechanisms organise selected episodes; they are not frequencies derived from such a coding pipeline or effects isolated by matched executions. The machine-readable index anc/case-evidence-matrix.json separates reported locators, author accounts and missing execution or release identifiers. Each null identifier marks an evidence link that still needs checking.

4 Exploratory testbed: observations

Table 2 summarises the case and Table 3 compares the periods before and during the delivery loop.

Table 2: Timeline of the case (all dates 2026).

Dates	Phase	What happened
11 Sep – 7 Oct	Research and requirements	Sixteen regulatory sources converted; role agents reject five end-to-end agents (26 Sep); BRD and PRD drafted and repeatedly revised; prototypes reviewed; participant usability feedback recorded (7 Oct).
8 – 14 Oct	A usable MVP	MVP built and reshaped through several feedback passes in one day (8 Oct); running on Azure with SQL, blob storage and two replicas over 980 synthetic cases (9 Oct); rebuilt with Entra ID sign-in, used by the participant (12–13 Oct); five assistant paths exercised against a real model (14 Oct).
14 – 15 Oct	Backlog and coordinator	Backlog generated as GitHub issues; coordinator created and hardened within a day; an ADR makes migrating the MVP to a new source layout the first priority.
15 – 22 Oct	The loop at scale	Pull requests merged at about eighteen a day; the log grows by about 37,000 words; the migrated deployment holds one fictional case, no staff and no model deployment (16 Oct); browser sign-in blocked; validation moved off hosted CI to save budget (18 Oct); cloud cost ceiling raised (22 Oct); new cross-cutting requirements appended to open issues (22 Oct).
22 – 24 Oct	Asking for something that runs	Ten issues labelled *feedback-first*; instruction to make employee and resident sites available as soon as possible with at most eight parallel items; checkpoint and refactoring instruction (24 Oct).

Table 3: Before and during the delivery loop. Counts from the repository at commit `29c6b51` and the GitHub API on 24 October 2026; evidence-boundary phrases as defined in Appendix A.6.

Measure	Before the loop	During the loop
Pull requests merged	52 (26 Sep – 14 Oct; 2.7 a day)	183 (15 – 24 Oct; 18.3 a day)
Knowledge-base log entries	126 (to 14 Oct)	242 (15 – 22 Oct)
Words appended to the log	19,967 to 14 Oct (about 1,000 a day from 25 Sep)	36,755 (15 – 22 Oct; about 4,600 a day)
Evidence-boundary phrases per 1,000 log words	1.2	3.8
Log entries citing the simplicity requirements	4 (last on 13 Oct)	0
Participant-usable deployment	Entra sign-in, 980 synthetic cases, live model calls (12 – 14 Oct)	One fictional case, no staff, no model deployment (16 Oct); sign-in blocked

4.1 Research, requirements and a usable MVP

Table 4 adds code-level activity from the pinned history. The script classifies paths and compares each selected first-parent commit with its first parent, detecting renames. The integration view includes multi-parent merges and PR-numbered squash commits: 31 before the loop and 177 during it. These are not the API pull-request counts in Table 3; the literal-merge subset, seven in each period, is also reported in `anc/repo_history_metrics.out.txt`. Classification rules and commands are printed there. These figures quantify editing activity. Net code size, maintainability and delivered value require different measures. The original analysis lacked complexity measurements. The post-hoc 1 November comparison below uses the kit's pinned static tools and does not execute historical product code.

Table 4: Text lines added and removed in first-parent integrations, including PR-numbered squash commits, through `29c6b51`. Detected pure renames add no lines; binary deltas are excluded.

Path class	8–14 Oct		15–24 Oct	
	Added	Removed	Added	Removed
Product source	8,136	421	23,408	2,184
Tests	6,871	537	73,478	1,416
Documentation	12,065	1,297	25,210	2,455
Tracking artefacts	238,185	2,642	240,369	1
Infrastructure	2,708	73	6,162	485
CI and workflows	240	64	897	246
Other	25,320	1,099	72,711	1,776

The seven-calendar-day MVP and ten-calendar-date loop windows differ from the nineteen-day pre-loop API PR window; the loop's last date is partial. Normalised by those calendar-date denominators, tracking additions fall from 34,026 to 24,037 lines/day, and product additions per integration fall from 262.45 to 132.25. The tracking gate formalised existing output, not its onset. Of the 72,711 "Other" additions, 60,001 belong to one CSV data asset; its contents were not inspected. The remainder includes evidence JSON, tooling, inventories and dependency manifests.

Table 5: Post-hoc static source snapshots, using Lizard 1.24.3, Ruff 0.17.3 with isolated settings and ignored suppressions, and jscpd 5.4.1. Source acquisition/composition changes alongside time.

Snapshot	Functions	Mean CC	Above CC 10	Duplication
14 Oct, `9ef7e40f`	767	4.03	7.82%	0.093%
24 Oct, `29c6b51`	1,895	5.24	11.82%	0.388%
29 Oct, `9b74ac8d`	2,561	5.12	11.79%	0.488%

Ruff findings per 1,000 Python lines fall from 188.22 to 142.30 to 125.83. Its submitted Python denominators are 6,365, 25,756 and 33,618 lines; JavaScript and TypeScript lines do not dilute this metric. Suppression-line counts across all selected source rise from zero to four to eight. These mixed proxies do not identify a causal decay rate or comprehension. The same source filter, exact commits, source hashes and commands are retained in anc/revision-metrics-2026-11-01.json. The corrected Python-only lint record is anc/revision-metrics-2026-11-01-python-lint.json; the original all-language density is retained as historical, superseded evidence. PR #445's complete diff is 116 additions and six removals across four files: production $+2/-1$, tests $+33/-2$, evidence $+78/0$ and score snapshot $+3/-3$. The small-fix comparison concerns production additions, not the entire change or an established net process cost.

The September work was research and requirements preparation, not application building or a project-free baseline. The agents assembled a regulatory corpus, sector baselines and a modelled customer profile, and the role agents produced a genuinely useful early verdict: each requested *end-to-end* agent bundled deterministic controls, accountable human decisions and a smaller set of tasks suited to bounded AI, so none should be built as such. Requirements were written, reviewed by an automated quality reviewer and revised many times. Application prototypes followed in October.

On 7 October the author used the latest prototype and recorded usability problems. The next day the agents combined those notes with critiques of three prototypes by the UX, Design Thinking and role agents into ten simplicity requirements (SA-001 to SA-010): a default view listing only the user's cases, each with one plain status and one next action; a case that opens

full screen; blockers explained in plain words with an owner; no role simulator in user views; optional AI that never blocks manual work; and so on. On 8 October they built an MVP that followed those requirements and then reshaped it through several rounds of participant feedback the same day: filters, an overview map, visits, handover suggestions and document uploads. On 9 October it ran on Azure with Azure SQL, blob storage and two replicas over 980 synthetic cases. On 12 October the stack was rebuilt from an empty resource group with Entra ID sign-in; the author reports signing in and using it. A missing logo and an account switch that only signed the user out were fixed within a day. On 14 October five assistant paths produced cited proposals through a real model deployment, using 6,884 tokens across five calls. The author tested an initial proof of concept for a couple of days, discarded it and used the MVP for a few days to refine ideas and the user experience. This was intermittent use, not daily use throughout the week. Those ideas informed the requirements, ADRs and hierarchical backlog.

The period 8–14 October supplies a within-case contrast with 15–24 October: intermittent use of a usable MVP versus no realistic end-to-end task completed in the replacement during the autonomous loop. The author continued handling approvals and technical blockers, but could no longer complete realistic tasks in the replacement. This contrast is consistent with the importance of feedback, but cannot establish that removing product-use feedback caused the outcome: the migration, workload, gate rules, models and deployment conditions also changed, and the periods were not assigned at random.

4.2 Bounded within-case contrasts

Table 6 compares episodes and negative cases already in the chronology. It is an analytical contrast, not a new PR-cohort study or an exhaustive search for disconfirming cases. In particular, documentation expansion preceded autonomy and the tracking/source churn ratio fell during the loop. Both limit a story in which autonomous coordination alone created the problem. The successful health and restart checks are also real successes; they did not exercise the same outcome as the blocked sign-in journey.

Table 6: Within-case contrasts and their limits, using the existing chronology and measurements.

Contrast	Recorded difference or negative case	What it can and cannot discriminate
Product use, 8–14 versus 15–24 Oct	Intermittent MVP use and same-day feedback versus no completed realistic replacement journey.	Consistent with feedback starvation; migration, environment, workload and models changed at the same time.
Requirements before versus during the loop	Growth and review overhead already existed before autonomous coordination.	Weakens an autonomy-onset explanation; there is no matched small-specification comparison.
Integration artefacts, 8–14 versus 15–24 Oct	Tracking/product-source additions fell from 29.28 to 10.27 while absolute output remained high.	Volume is not a monotonic decay measure; churn does not assess understanding or usability.
Replacement engineering checks versus user task	Health, restart and unauthenticated-request rejection passed; real-user sign-in remained blocked.	Shows the boundary of those checks, not that passing checks were fabricated or that intrinsic code quality declined.
Domain challenge versus backlog execution	Early role agents rejected the five-agent proposal; later sessions delivered the specified issues.	Negative case against inevitable agent compliance; role, task and human interaction differ, so this is not a treatment comparison.

4.3 The backlog and the coordinator

Late on 14 October the backlog manager generated the backlog from the BRD, PRD and ADRs. Many issues carried a label stating that part of the work already existed in the MVP. On

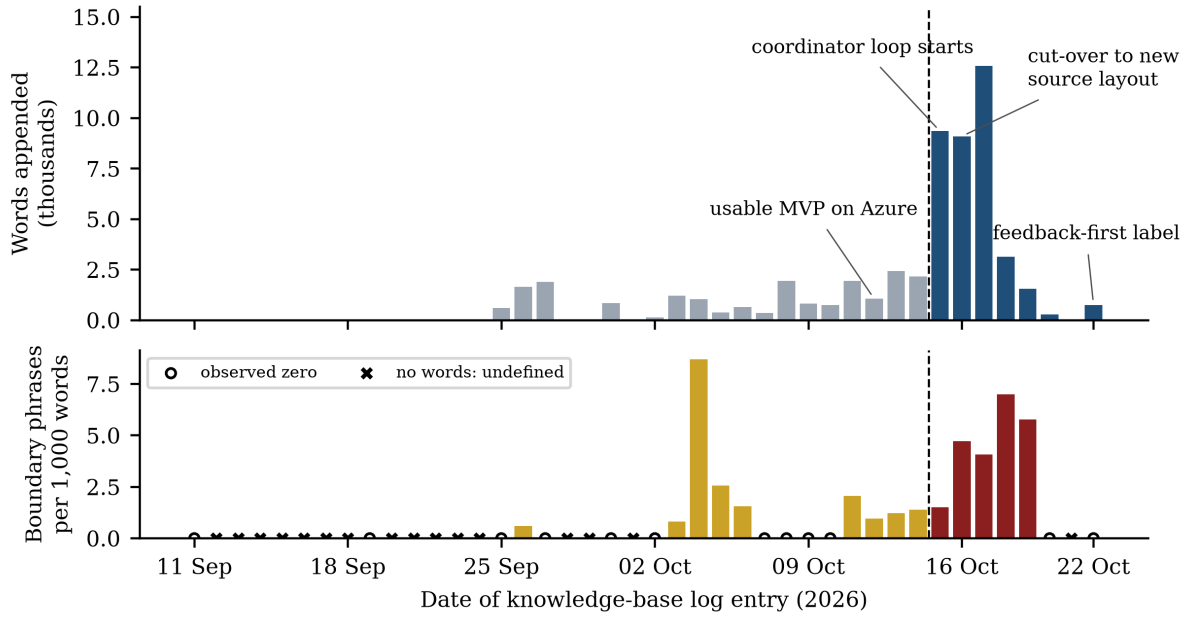


Figure 4: Words appended to the knowledge-base log per day (top) and lexical evidence-boundary matches per 1,000 words that day (bottom), from the packaged daily data. Circles mark observed zero; crosses mark days with no words and undefined density. The full plotted window starts on 11 September; pre-25 September entries are reconstructed. The dashed line marks the coordinator's start on 15 October 2026.

15 October the author created the coordinator and, as Section 3.3 describes, hardened it within the day. The log recorded 91 entries that day, against at most 19 on any earlier day. The accepted source-layout ADR made the migration of the MVP's code the first priority; the MVP itself was to be retired.

4.4 The loop at scale

From 15 to 24 October, 183 pull requests were merged, compared with 52 in the nineteen days before. The knowledge-base log grew by 36,755 words between 15 and 22 October, 31,005 of them in the first three days (Figure 4). The density of lexical evidence-boundary matches, such as "remains unmet", "not inferred" or "not accepted", rose from 1.2 to 3.8 per thousand words.

The pre-loop density rests on 24 matches, nine on 4 October. The inspected, seeded twenty-match sample contains twenty explicit evidence-status/approval boundaries; this assisted lexical precision check is not independent human coding or factual accuracy. The day-one rule to tick only proven criteria also changed reporting conventions.

By 16 October the recorded replacement already lacked staff, model and registration prerequisites; issue #349 was opened at 16:34:30 UTC. That is a dated blockage record, not the first actual failed-sign-in timestamp, which remains unavailable. The migration succeeded on its own terms. A private deployment built from the new source layout passed health checks, rejected unauthenticated requests and survived restarts. But it held one fictional case, no staff accounts and no model deployment, against the 980 cases, eleven staff identities and five working assistants of the system it replaced. A public entry point was restored through an additional edge service, yet real-user sign-in stayed blocked because the new deployment had neither an application registration nor staff bindings, the very things set up and verified on 12 October. Recovering them became a chain of issues, each with its own research, plans, reviews and corrections. To preserve the hosted CI budget, validation moved to a local gate on 18 October (issue #413). The demonstrator's cloud resources accrued USD 189.94 between 1 and 20 October, with

a forecast above the USD 500 owner-set ceiling; on 22 October the author raised it to USD 700, deleted a test edge profile and scaled a demo service to zero. The same day, after a mentor raised the economics of AI consumption, the agents produced new BRD and PRD versions and appended a new set of acceptance criteria to fifteen open issues.

A Copilot usage dashboard captured on 24 October showed USD 1,809 of price-list consumption value, 10,229,625,825 tokens and 100,938 requests in the rolling last 28 days. These were account-wide totals across all work, not AMA-only measurements or an invoice. The token total included 10,081,179,684 cached tokens. Surface and model breakdowns described the same total and must not be added together; the requests were not established as premium requests. The screenshots do not isolate 15–24 October, establish exact window boundaries or support an estimate of the loop's incremental cost.

4.5 Asking for something that runs

On 22 October the author requested a usable increment. A *feedback-first* label, described as prioritising "the bounded usable synthetic demo slice", was applied to the relevant issues, ten of them by 24 October. On 24 October the author requested employee and resident sites to be available as soon as possible, with at most eight items in parallel. These instruction dates and the eight-item cap are author-confirmed recollections, not quotations recovered from the pinned log. The agents created the smallest independently runnable resident preview they could design (issue #444): read-only, synthetic and bound to the local machine, with thirteen acceptance criteria. The same day a two-line fix to the rubric scorer arrived with a 78-line evidence record and an explicit, one-time owner exception to interrupt the full local gate (pull request #445). Following Beck's advice to alternate features with consolidation, the author also instructed the coordinator to stop at checkpoints, review what had been built, exercise the functionality and refactor. The outcome of that instruction is outside the case's evaluation boundary.

4.6 Dated repository follow-up: 29 October

The original case ends on 24 October; it is not silently extended by later repository activity. A separate refresh on 29 October, captured between 17:38:52 and 17:39:47 UTC, queries GitHub metadata and measures the default branch at `9b74ac8d`. This is a repository-status observation, not a new deployment test or an experimental result. Table 7 summarises the snapshot.

Table 7: Repository follow-up captured on 29 October 2026. GitHub counts cover all repository branches; document and log measures use pinned default-branch files. Counts and scores do not establish accepted behaviour.

Measure	Current observation
Merged pull requests	314 overall; 262 since 15 Oct
Post-case merges	79 after the pinned case integration; 75 during 25–29 Oct
Issue states	211 issues: 136 open and 75 closed
Other PR states	11 open; 19 closed without merging
Current log entries	458 dated entries; 69,352 entry words; 207 lexical evidence-boundary matches (2.98 per 1,000 entry words)
Whole-file document sizes	Index 12,535; log 69,465; BRD 43,410; PRD 48,094 words
Combined document volume	173,504 whole-file words across the index, log, BRD and PRD
Retained automated rubric score	77/100; implementation completeness remains 5/5, on the scorer's recorded commit

The merge boundary matters. The pinned case commit `29c6b51` has a Git timestamp of 24 October at 16:19:38 UTC, while GitHub records its pull request, #445, as merged at 16:19:39.

Counting from 15 October through that integration reproduces the reported 183. Stopping at the Git timestamp instead gives 182; including the whole 24 October UTC calendar day gives 187. The historical count is therefore retained with its integration boundary made explicit, rather than replaced by the whole-day count. Git commit times, PR merge times and retrieval times are different measurements.

Using the original first-parent integration heuristic and path classifier, the period after the pinned case commit through `9b74ac8d` contains 71 integrations. Product-source lines added/removed are 11,907/844; tests 31,126/478; documentation 17,338/558; and tracking artefacts 181,261/34. Tracking additions are 15.22 times product-source additions in this follow-up. These are editing volumes, not a quality trend; the 71 first-parent integrations are not the 79 GitHub merges across all branches.

The log counts parse the current file using the original entry and lexical rules. Entry words exclude its preamble; whole-file counts include it. Earlier-dated records in the current file need not match the earlier pinned version, so differences are not labelled newly appended entries. The combined document volume is not a measurement of material actually loaded into any agent session.

The BRD and PRD grow from 37,896/33,852 words at the case pin to 43,410/48,094, approximately 15%/42%; combined index/log/BRD/PRD volume rises from 138,806 to 173,504, approximately 25%. The 27 October owner sign-off authorised directory retirement, not new soak or deployed-journey acceptance.

The retained score was recorded on 29 October at 17:12:55 UTC for `0717be24`, not freshly recomputed for the current default-branch commit. Its 5/5 completeness rating remains an automated repository assessment, not acceptance of a deployed user journey. No live sign-in or end-to-end task was executed for this refresh; neither restored usability nor continued failure on 29 October is established. The raw selected metadata, document hashes, daily counts, classified churn and score provenance are retained in anc/repository-metrics-2026-10-29.json.

5 Five candidate mechanisms

Five mechanisms organise the gap between a pull request merged roughly every eighty minutes and the absence of a usable replacement. Table 8 summarises them; the supplementary case audits sets out a definition, indicators, counter-evidence, rivals and testable implication for each. Together they connect the inherited-state problem to the way this delivery loop selected work, carried lessons and checked outcomes.

They occupy different points in the delivery process (Table 9). Specification growth increases the obligations and interactions that later work must preserve. Learning evaporation concerns the path by which a relevant lesson reaches the next session. Proxy capture concerns what selects and approves work; process-gate inflation and feedback starvation concern the capacity to detect and correct cumulative problems. None needs to be called error propagation to explain how an inherited fault can survive.

Until 24 October the loop had no scheduled cumulative consolidation step. Its large restructuring, the source-layout migration, was planned as backlog items; local refactoring could still occur within individual issues. The experiment separates retained-contract loss, structural fidelity diagnostics and later adaptation. Formal fault-lineage annotation would address the stronger question of whether an earlier fault generates distinct later failures; it is deferred rather than added to the pilot's workload.

The five categories in Table 8 are post-hoc interpretations. The supplementary case evidence retains their detailed arguments, counter-evidence and discriminating tests. They are not measured frequencies or independently identified causal effects.

Table 8: Five candidate mechanisms, case indicators, relevant prior work and proposed early-warning signals. Indicators are not causal tests.

Candidate	Case indicators	Related prior work	Early-warning signal
Specification inflation	BRD ≈37,900 and PRD ≈33,900 words; 192 issues; 17 criteria on one issue and 13 on the smallest preview	Royce; Gall; Brooks; Böckeler; Beck	Specification growth versus human-validated behaviour
Proxy capture	Artefacts made a merge gate; 78-line evidence record for a two-line fix; score 5/5 while sign-in is blocked (release/scope linkage pending); boundary-phrase density tripled	Goodhart; Strathern; Beck's genie; ImpossibleBench; METR	Artefacts per behaviour change; scores that disagree with a live demo
Learning evaporation	Lessons not carried in the described briefs; last cited 13 Oct; read path ≈139,000 words	Naur; context studies; multi-agent failure taxonomy	Lessons that exist only as prose; problems solved twice
Process-gate inflation	Full gate on every push; CI minutes rationed; exception for a two-line fix; 18 merges a day delegated	Bainbridge; ironies of generative AI; Murphy; Goldratt	Gate time per change; review minutes per merge
Feedback starvation	Working MVP frozen; replacement unusable after ten days; request for a demo turned into more issues	XP; incremental delivery; tracer bullets; Agile Manifesto; DORA	Time to first human use; age of last usable increment

Table 9: Accumulation, transmission and correction in the working model.

Process	Question	In this case
Accumulation	What obligations and change burden does each feature leave behind? [58, 59]	No scheduled cumulative consolidation until 24 Oct; restructuring planned as backlog items
Transmission	Which implementations, rules and lessons shape the next change? [78, 92]	Growing specifications and read path; previously working identity setup not preserved
Selection and correction	Do review and tests expose the cumulative consequences before more work depends on them? [28, 102]	Proxy capture; process-gate inflation; feedback starvation

5.1 A proposed reinforcing relationship

The mechanisms suggest a possible reinforcing loop (Figure 5). In this case a large specification accompanied a large backlog and parallel sessions. Artefact-based gates generated more documents; some recorded lessons were absent from later briefs; regressions were not exposed through routine product-use feedback. The author responded to gaps with more rules, criteria and gates. That sequence is consistent with reinforcement, but does not establish each causal link. A running increment and a human checkpoint are proposed conditions of every cycle, to test whether they interrupt it.

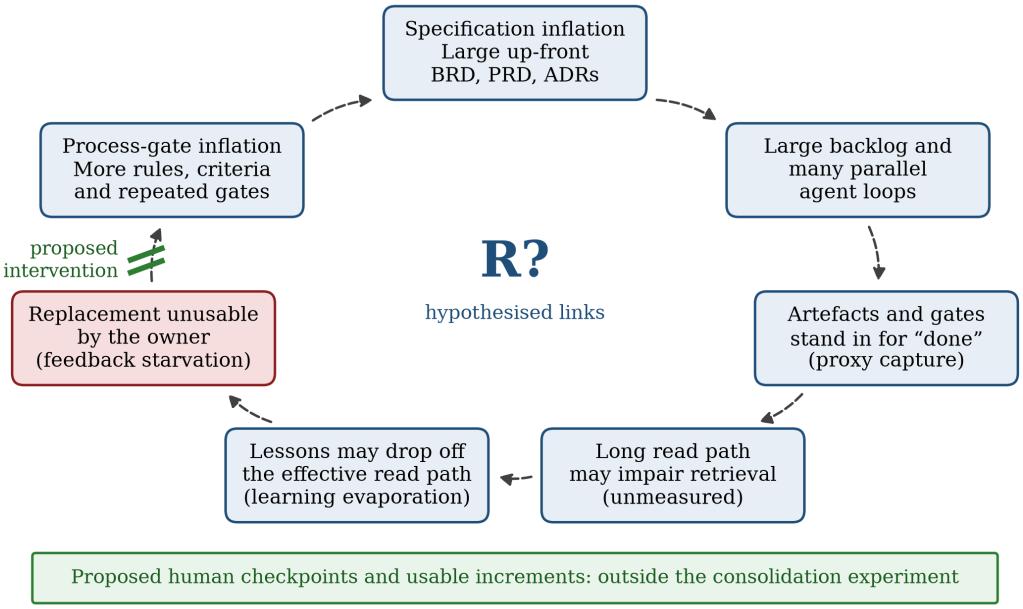


Figure 5: Proposed reinforcing relationships drawn from the case interpretation. Their causal effects remain unmeasured. Context overload is unmeasured. The proposed human-checkpoint intervention differs from the consolidation package tested in Section 7.

5.2 Case validity and rival explanations

The five categories are not independent causes identified by these data. The same missing identity and staff bindings can indicate deferred migration work, incomplete deployment acceptance or a lost lesson. Discriminating them requires the deployment comparison and actual session-input trace identified in Table 1, not another count of documents or citations. Similarly, a growing read path does not demonstrate context overflow, and an expensive gate is not necessarily a disproportionate gate. Per-change gate time, avoided defects and human effort were not measured.

The source-layout migration, owner-selected priorities and delegated merging changed alongside coordination. Scope additions, budget constraints, model/tool changes and deployment conditions also differ between periods. These are direct rivals to an autonomy-only account. The supplementary rival-explanation register retains the full register, separating recorded circumstances from plausible but untested alternatives. Bird et al.'s study of co-installed applications on Windows machines supplies a methodological analogy for checking environmental influences on reliability [19]. It neither tests AI-agent configuration nor diagnoses this case's sign-in failure.

Construct validity is limited by whitespace counts, lexical indicators and selectively reviewed records. Appropriate caution can raise boundary-phrase density; missing references do not prove missing knowledge. As coordinator designer, participant and analyst, the author selected the episodes and may favour a coherent retrospective explanation. AI synthesis is not an independent observer. Public records support recomputation but not independent corroboration of every statement. The single fictional-domain capstone and purposively selected external examples do not establish prevalence or general causal effects. The proposed experiment has separate identification limits in Section 7.3.

6 Consolidation hypothesis and harness

The observed operational failure and the prospective hypothesis are distinct. Missing deployment identity and staff bindings are concrete obstacles to sign-in; structural code consolidation would

not necessarily restore either. Recovery of that journey requires configuration and end-to-end verification. Consolidation is worth testing for a different reason: repeated feature work may make later changes harder, even when deployment dependencies are correct. The experiment probes that possibility through retained tests and CR1; it does not claim that consolidation would have prevented this episode.

The idea is already practised. OpenAI describes replacing weekly manual clean-ups with recurring background agents that open small refactoring pull requests, alongside a documentation-gardening agent [62]. Anthropic asks each session to leave a clean state and update a progress file [101]. Neither account reports a controlled comparison of feature-only delivery with consolidation between feature steps. Darwin's continuous optimisation is a further precedent for automated improvement, but its performance evaluations do not isolate that intervention or test an unforeseen change request [81] (Section 2.3). RefactorBench already evaluates refactoring agents [40]; the narrower hypothesis concerns retained behaviour and later adaptation during repeated feature additions.

AutoSaddler's preprint abstract already treats harness improvement as offline learning from execution traces, combining failure diagnosis, structured patches and validation-based update selection [79]. Agent-led reflective correction is therefore not the novelty claimed here. The proposed intervention instead consolidates the retained repository between feature steps and measures retained contracts and a withheld change request.

The proposed consolidation harness tests a bounded part of reflective correction. The distinction is what persists afterward. Self-Refine revises an output and Reflexion retains lessons for later trials [65, 90]; the present intervention instead consolidates the software and its handover between different feature requests. CRITIC's tool feedback is another precedent [44], but it does not authorize feeding the held-out evaluation into this experiment. Whether the retained changes help an unforeseen request remains the empirical question. The harness schedules each pause; the agent is not authorised to reject features or redirect the product. After every feature the loop stops, and an agent:

1. runs the tests and a metrics script that lists complexity hotspots, duplicated blocks and lint warnings;
2. reviews the last feature's diff and the hotspots against a short checklist (duplication, long or complex functions, unclear names, dead code, rules from one feature leaking into another, tests that check implementation rather than behaviour);
3. refactors in small, behaviour-preserving steps, each applied through `test && commit || revert` [10], so that every change either keeps all tests green or disappears;
4. may add tests that pin current behaviour, but may not change or delete the example tests that define the features;
5. rewrites a design note of at most 300 words that says where each rule lives and what debt remains, replacing the previous one rather than appending to it;
6. stops when a quality budget is met or its time budget runs out.

Only then may the next feature start. The supplement reproduces the historical v4 prompt; the current researcher kit defines the amended prompt. The intervention combines refactoring, additional tests and a design-oriented note; it is not a pure refactoring treatment.

The hypothesis draws on Beck's alternation of features and futures [13, 16] and *Tidy First?* [11]. It is not free, and may not work. Agent refactorings can be incomplete or incorrect [40]; human studies also caution against equating a refactoring label with a quality gain. Kim et al. report developers' refactoring challenges and benefits [54]; Bavota et al. find that observed refactorings do not consistently target or remove code smells [5]. These findings motivate checking outcomes rather than assuming improvement. Test-commit-or-revert protects

only behaviour represented by its tests, not every intended contract [64]. Section 7 specifies how the package will be evaluated.

7 Testing the harness: a prospective experiment

The experiment asks whether scheduled correction improves the repository inherited by later feature work. RQ3 is prospective: this article reports the motivating case, not an answer from collected trajectories. The 1 November amendment supersedes v4's two runs per arm/model. Historical freezes, diagnostics and the original owner-reported OSF registration are retained; none approves this new design. The researcher kit requires a new source-bound freeze and prospective OSF amendment before collection. No experimental results are reported. Agent configuration protocols and empirical-study guidance inform this pre-data design [4, 63, 98].

7.1 Design and estimands

One independent replicate is a fresh repository trajectory through S1's ten ordered features and a final held-out change request, CR1. Every selected model, GPT 6.1 Sol, Sonnet 5.5 and Gemini 4 Argon, runs every arm. Randomised interleaving within model/replicate blocks uses seed 2027. Feature steps, tests and checkpoints are repeated observations within a trajectory, not independent replicates.

Table 10: Prospective five-arm allocation. D is the matched-resource A+ control; E tests visible regression repair, not hidden-test feedback.

Arm	Intervention and information	Per-cycle ceilings
A	Feature delivery only	30 min; 300 nominal credits
B	Feature, then behaviour-preserving consolidation, additional tests and replacement DESIGN.md	30 + 15 min; 300 + 300 credits
C	Feature, then read-only review and replacement REVIEW.md	30 + 15 min; 300 + 300 credits
D (A+)	Longer feature delivery, without a scheduled consolidation pause	45 min; 600 credits
E	Feature, then repair of previously specified behaviour using visible specifications and examples	30 + 15 min; 300 + 300 credits

B–D is the primary practical comparison: consolidation versus giving the builder the same total time and nominal credits. Equal ceilings do not guarantee equal realised resources; D also has fewer fresh sessions. B–A measures the additional package; B–C also changes editing rights, tests and handover content. E–B contrasts visible behavioural repair with behaviour-preserving consolidation. These secondary package comparisons do not isolate refactoring or any of the five case mechanisms.

C cannot repair code during its review session, but REVIEW.md can direct repairs in later feature sessions. Follow-through needs review-item and diff locators with external adjudication; file overlap alone is not proof. B's added tests must agree with visible specifications, rather than legitimating a defect. Subsequent feature sessions must keep accepted tests green. E can correct an agent-authored expectation only with a specific contradictory specification and a retained explanation. Accepted Python test files are immutable in full, including imports, fixtures and module/class constants; additions use new files. E retains only its explicit specification-bound correction exception. Immutable example tests remain protected in every arm.

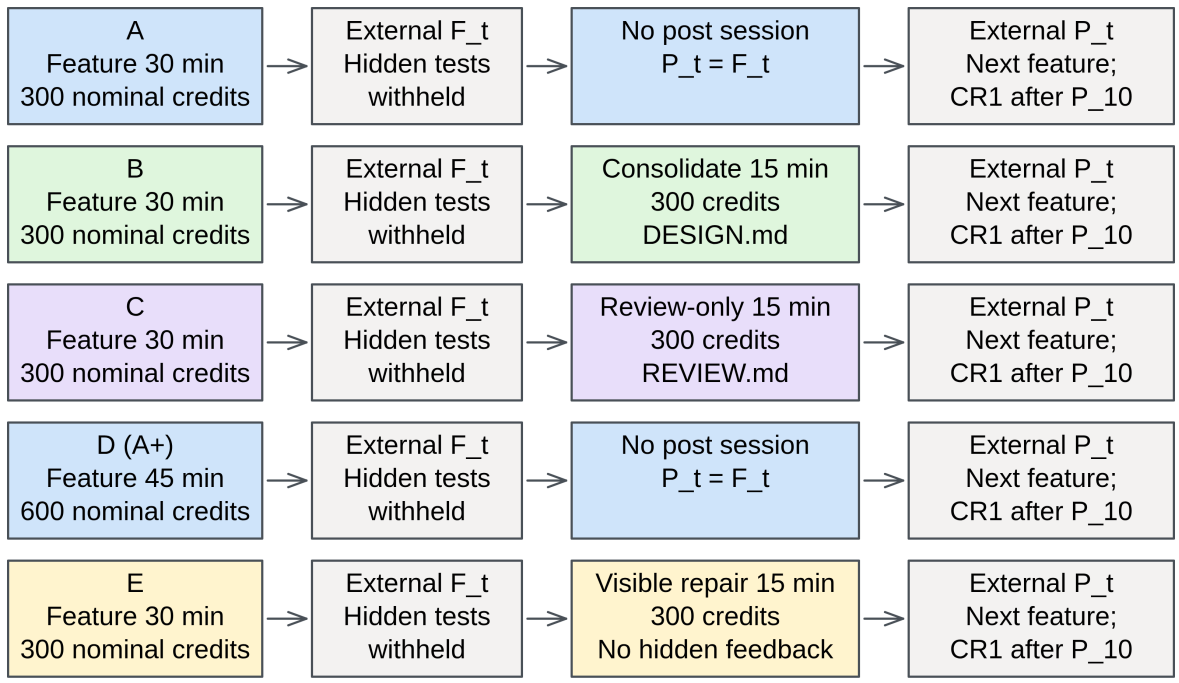


Figure 6: Prospective design, not results. Every model runs all five arms. The external evaluator is separate from candidate tools. CR1 content remains withheld until its session; privacy and isolation reduce exposure without proving that a model has no prior knowledge.

7.2 Outcomes and retained-contract loss

The primary outcome is final behavioural coverage: contracts passed out of all 127 scheduled feature contracts at P_{10}, before CR1. Coverage combines acquisition, retention and repair; it is not itself decay. Retained-contract loss and CR1 adaptation are distinct secondary outcomes, with legacy identifiers H2 and H6 retained only for data traceability. HIR/IDR definitions are in the supplementary appendices; comprehension is not measured.

Let $\mathcal{R}_{r,t}$ be eligible unchanged contracts passing at an earlier end-of-cycle checkpoint P_u, $u < t$. At F_t and P_t, let $\mathcal{O}^p_{r,t} \subseteq \mathcal{R}_{r,t}$ contain contracts with observed binary results $S^p_{r,t}(q)$, $p \in \{F, P\}$. Report

$$L^p_{r,t} = \sum_{q \in \mathcal{O}^p_{r,t}} \mathbf{1}\{S^p_{r,t}(q) = 0\}, \qquad d^p_{r,t} = \frac{L^p_{r,t}}{|\mathcal{O}^p_{r,t}|}. \tag{1}$$

An empty observed denominator is undefined. Publish the full risk set, observed subset and missing count alongside the density; incomplete observations do not identify the full-risk loss count. Never-acquired contracts are acquisition failures, not losses. Retain onset, recovery, recurrence and uncertainty across missing checkpoints. Recovery does not erase an episode.

Distinguish same-cycle post-step losses and rejected attempts from losses in retained code.

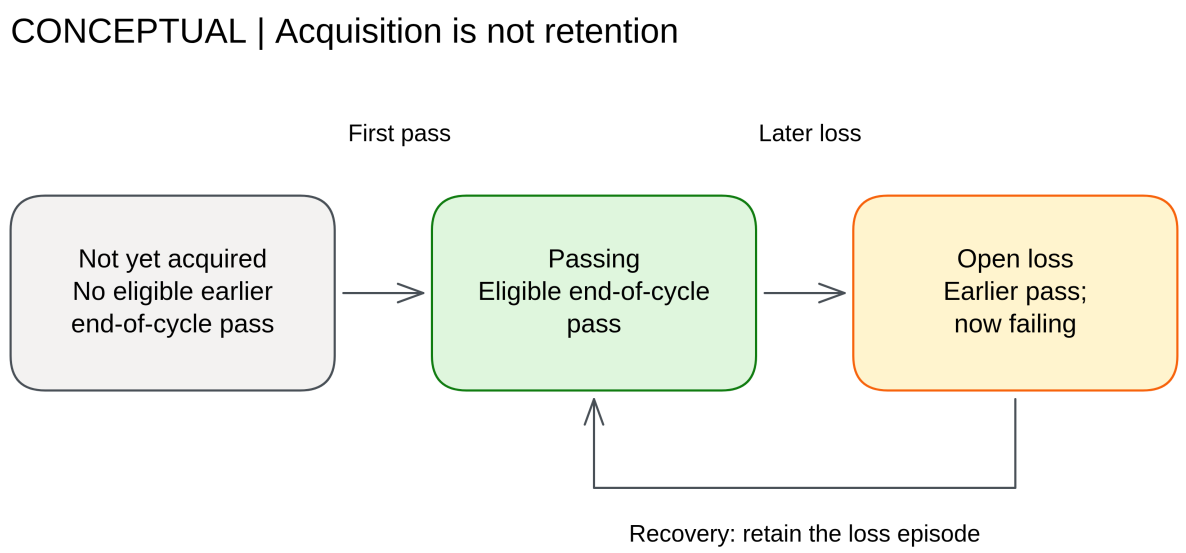


Figure 7: Conceptual contract states. Acquisition, loss, recovery and unobserved evaluation are distinct; the diagram contains no measured frequencies.

CR1 success requires all 15 CR1 contracts and retention of every feature contract passing before CR1. Also report contracts passed out of 15, previously passing contracts lost, and observed/missing denominators. Stratify by whether F9 and F10 prerequisites were acquired before CR1: an incomplete arm has fewer retention obligations, and prerequisite strata are post-treatment selections, not randomised subgroups. One fixed CR1 is task-specific. A prewritten, randomly selected request pool belongs to a separately registered subsequent study.

Actual AI credits include every delivery and CR1 attempt, including failures, assistance and infrastructure retries; phases are reported separately. Cheap failure is not efficiency. Credits, tokens, elapsed time and currency are distinct. Lizard measures complexity, Ruff lint and jscpd duplication. External fixed settings and `ruff -isolated -ignore-noqa` prevent local suppressions from silently improving the lint score; count added suppressions separately. SDK request wall time and tool time are observable. Provider-active time and rate-limit waits require explicit telemetry; absent telemetry is not zero.

7.3 Replication and uncertainty

The author selected a minimum useful absolute coverage gain of ten percentage points. This is approximately thirteen test-weighted contracts, not thirteen independent capabilities or a threshold prescribed by Beck. The prospective planning assumption is a within-cell standard deviation of 25 points, not an observed estimate. For one two-sided B–D comparison, $\alpha = 0.05$ and target power 0.80, the normal planning approximation is

$$n_{\text{cell}} = \left\lceil \frac{2(z_{0.975} + z_{0.80})^2 \sigma^2}{3(10)^2} \right\rceil = 33.$$

This gives 99 trajectories per arm, 495 overall. The target power is conditional on independent trajectories, the assumed variance and the blocked estimand; it is not guaranteed. SD assumptions of 15, 20 and 50 points require 12, 21 and 131 per cell respectively. The bounded score's

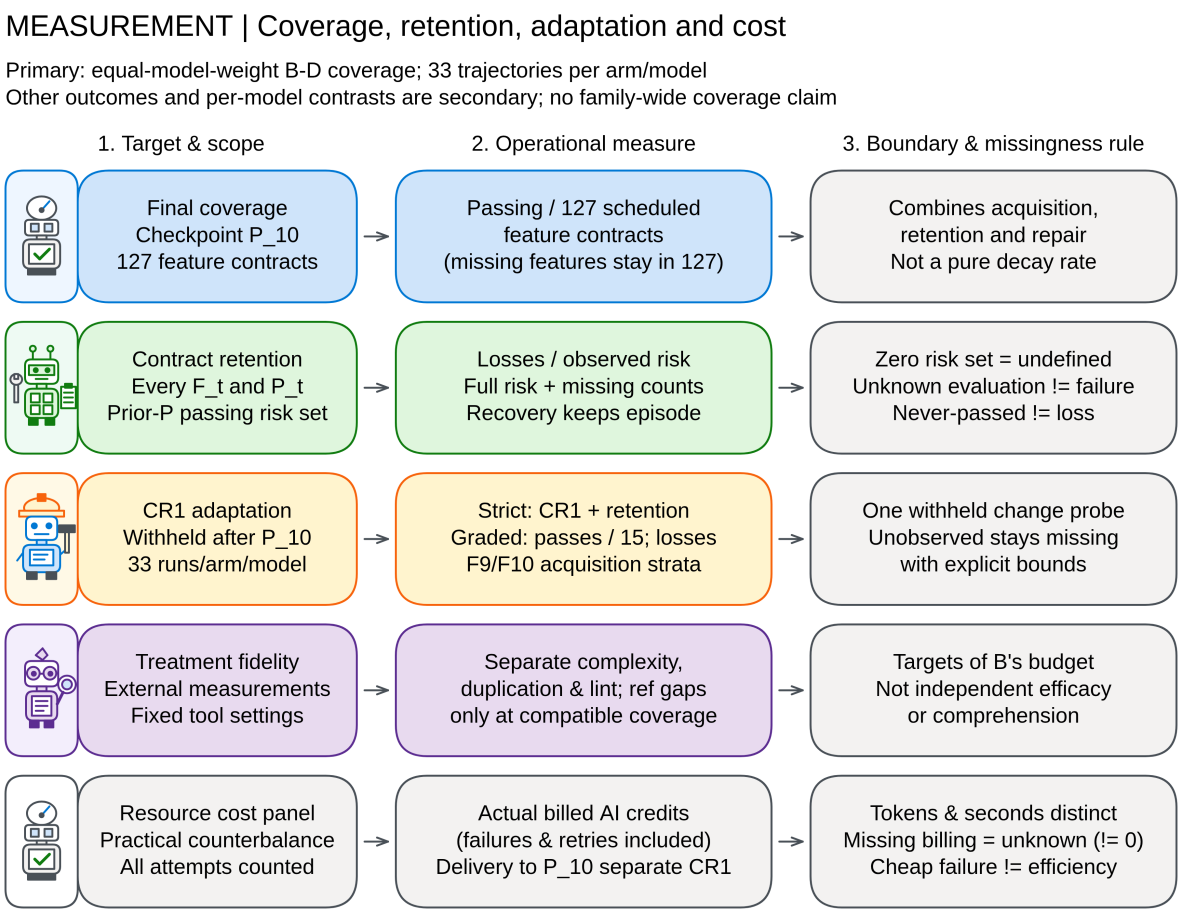


Figure 8: Coverage, retained behaviour, adaptation and all-attempt cost are separate measures. Structural metrics targeted by B's stopping budget are treatment-fidelity checks, not independent efficacy outcomes. Unknown results and billing remain unknown.

worst-case SD is 50; the 25-point assumption needs explicit author and prospective registration approval.

The primary estimate equally weights the three within-model B–D mean differences. Resample whole trajectories within model and arm, with 10,000 draws and pointwise percentile intervals; disclose missing frames rather than silently pool observed models or complete cases. Model-specific estimates and other contrasts are secondary, with no family-wide coverage claim. Chen, Zaharia and Zou's dated ChatGPT comparison motivates monitoring service drift despite exposed names [25]; it does not measure drift in this roster or make a version pin an immutability guarantee. Report the pooled within-cell SD with its degrees of freedom for future planning. The old two-run percentile bootstrap cannot justify a nominal 95% coverage claim: show its four pairwise differences instead. With v4's extreme CR1 split, 2/2 versus 0/2, the conservative difference interval is approximately $[-0.78, 1]$; pooling six per arm still gives approximately $[-0.04, 1]$. The coverage calculation does not power binary CR1; report exact binomial component intervals with their assumptions.

7.4 Resources, feasibility and collection gates

The unchanged envelope is 300,000 nominal credits, with 270,000 local admission, prior spending and unknown historical provisions retained. Original 300-credit sessions use a 270-credit soft limit; D's 600-credit feature uses 540. A retry shares the original allowance. In-flight responses can overshoot; these are not provider billing caps. No verified dollar conversion is available.

Table 11: Planned resources, not expenditure. Worst-case allowances exceed the aggregate envelope; affordability is a real launch gate.

Arm	Trajectories	Sessions	Maximum agent-hours
A	99	1,089	569.25
B	99	2,079	816.75
C	99	2,079	816.75
D	99	1,089	816.75
E	99	2,079	816.75
Study total	495	8,415	3,836.25
Excluded calibration, no CR1	15	240	105.00

At nominal caps the combined allowance is 2,902,500 credits, not a permitted expenditure. Cost-only excluded calibration must show that the entire fixed allocation fits remaining admission and the deadline; otherwise halt and amend before collection, never quietly reduce the sample. Proposed progression criteria are infrastructure failures at most 5%, no missing primary observations, complete billing/model/help records, zero oracle contradictions in triplicate reference checks, and no information-boundary failure. Hidden-test mutation assessment must report its actual mutant denominator and unresolved equivalence. These are feasibility criteria, inspired by pilot-study guidance [35], not a clinical CONSORT compliance claim.

Only current and previous visible specifications and examples reach candidate tools. Hidden tests, scores, identifiers, messages, reference code and future CR1 remain external. Candidate commands use no-network containers; the trusted host SDK's provider connection is different. The independent researcher receives a protected kit without access to the original repositories or manuscript. Use a researcher-managed workspace/account without the author's original checkouts, manuscript or session-context caches; record actual access and prior exposure rather than assuming blindness from a private flag. The owner will make both repositories private before collection and release approved materials with the paper afterward. OSF registration/embargo and GitHub privacy are separate; neither removes prior public or model exposure. Fresh source-bound access checks, approval, freeze, prospective OSF recording and launch permission remain required.

Unsuccessful feature increments are reset; later features and scheduled B/C/E post-steps continue on retained green state. Hidden denominators still include undelivered features. One infrastructure-only retry shares the original budget; poor performance never justifies replay. Contradictory valid oracle results halt collection. Assisted and incomplete trajectories and all their costs are retained, without replacements or success-conditioned primary analyses.

No study results are available. PF1 will show coverage and retention; PF2 graded/binary CR1, prerequisites and costs; PF3 coverage against all-attempt credits. No illustrative data or fitted frontier is substituted.

8 Discussion

8.1 Projected cost of omitting consolidation

The experiment measures costs over ten features and one final change. It does not yet estimate the price of maintaining a larger system without refactoring. Figure 9 and Table 12 therefore give an explanatory sensitivity scenario, not results, an empirical forecast or an addition to the frozen analysis. The economic hypothesis is that early investment can reduce the implementation and repair burden inherited by later features. It can fail: refactoring may consume credits without reducing that burden or may damage behaviour. Cost must always be read beside coverage, retention and adaptation.

Let N be successive feature cycles, k the assumed average consolidation investment in AI credits per cycle, and $\delta \cdot (t-1)$ the assumed excess feature-implementation cost in A relative to B at cycle t, excluding B's consolidation session. Shared baseline implementation costs cancel. Under the explicit simplifying assumptions of constant k, a linearly growing relative premium $\delta \geq 0$, and comparable delivered behaviour, the cumulative net difference is

$$\Delta C(N) = C_A(N) - C_B(N) = \delta \frac{N(N-1)}{2} - kN. \tag{2}$$

The first term is the assumed gross avoidable burden of omitting consolidation; the second is the investment to avoid it. A positive difference means A costs more under these assumptions, not measured savings. For $k > 0$ and $\delta > 0$, break-even is the smallest integer $N \geq 1 + 2k/\delta$; when $\delta = 0$, the investment never pays back in this credit-only model. Real trajectories need not follow a linear premium, and B can retain debt too: δ is a relative gap, not A's absolute debt or a claim that B is debt-free.

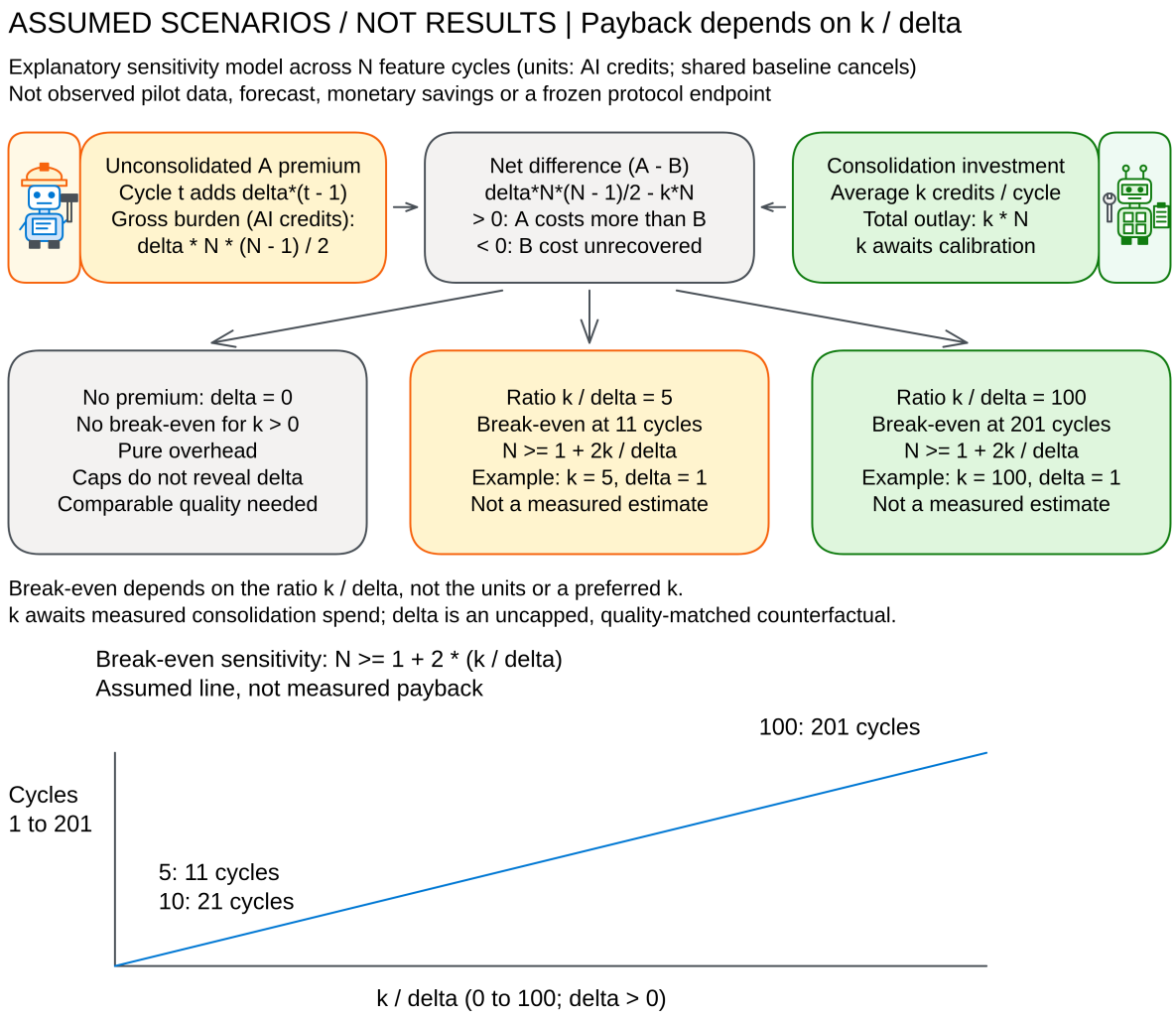


Figure 9: Assumed cost-of-inaction mechanism and break-even sensitivity, not pilot results or an empirical projection. A linearly increasing relative implementation premium accumulates across feature boundaries; recurring consolidation has an explicit cost. Payback depends on k/δ: a ratio of 5 breaks even at 11 cycles and a ratio of 100 at 201; $\delta = 0$ gives no payback for $k > 0$. Parameters are invented for transparent sensitivity, not fitted to study data. No dollar conversion or human-work estimate is implied.

The study can report actual B consolidation consumption and all-attempt CR1 cost by model. Calibration can anchor k once billed spend exists. Because feature budgets are capped,

Table 12: Illustrative net A–B cost in AI credits from Equation 2, with assumed $k = 5$ credits/cycle. Negative values mean B's investment has not paid back; zero and positive values mean break-even and a conditional credit advantage. None of the scenarios is an observed or preferred estimate.

Assumed premium increase δ	10 cycles	20 cycles	40 cycles
0 credits per successive cycle	-50	-100	-200
0.5 credits per successive cycle	-27.5	-5	190
1 credit per successive cycle	-5	90	580

harder work can instead reduce coverage: δ is an uncapped, quality-matched counterfactual, not a directly identified endpoint of this design. Even the enlarged replication over ten features does not identify a stable long-run δ or justify extrapolation to 40 features. A future forecast would require longer replicated sequences, matched delivered quality, measured residual repair work and independently checked billing. Human review, operational failures, opportunity costs and currency prices are not measured by this kit and cannot be monetized from its credits or code-size proxies. Thus the current projected cost of inaction is conditional and unestimated, not a claimed business saving. For comparison, assumed $k = 100$ and $\delta = 1$ yield break-even at 201 cycles, not eleven. No observed consolidation spend supports a preferred k yet.

8.2 What worked

The case is not evidence that agentic delivery is inherently ineffective. The research phase assembled a corpus and analysis in days; a human-only duration was not measured. The role agents' challenge killed a bad idea, five end-to-end agents, before any code was written. The agents refused to invent approvals, figures or company facts, and recorded the limits of their evidence. The MVP was built and reshaped in a day. These episodes included targeted human challenge and feedback; they are not a controlled comparison with human-only teams or proof that all short loops succeed.

8.3 Human judgement and reflective work

The participant supplied several kinds of reflective work: choosing what not to build, recognising the unusable replacement and deciding which earlier lessons mattered. Naur's theory-building account helps describe that retained understanding [74]. Beck's distinction between visible output and understanding supplies another framing [15]. These judgements were supplied by the author during the case. Whether agents could make comparable judgements remains open. The narrower experimental question is whether bounded post-feature correction preserves behaviour and later changeability.

Human sign-off also need not establish who formed a judgment. In Anthropic's randomised study, 61 analysts were recruited and 50 used the agent. Those 50 analysts matched their agent's randomly assigned preference on 86% of four analytical choices [45, Study design; Results]. The preferences were deliberately strengthened in one farming-analysis task with one model; the rate is not an estimate for coding or Zava North Homes [45, Our study's limitations]. This challenges an assumption that retaining approval alone preserves independent judgment, not the case's reported decisions. The article proposes asking experts for their view first or making the best case for each option [45, What this means]. Separately, recording alternatives and a human's reasons before accepting an agent's recommendation is the present author's proposed safeguard, not a remedy evaluated in that study.

8.4 Can another agent restore futures?

Beck's alternation suggests preserving the options consumed by feature execution [13, 16]. Darwin, AutoSaddler and SkillOpt already provide precedents for agent-guided correction; reflection itself is not the proposed novelty. Ng's reflection framework also argues for evaluating and revising work rather than treating first-pass generation as the finished product [75]. Improvement here means a measured advantage in retained behaviour or later adaptation, not merely writing a lesson, changing a prompt or calling the process self-improving. The experiment does not train model weights; the evolving repository and handover are the states under test. An exposed model name is not a guarantee against provider-side changes.

The memory presentation's proposed "dreaming" pass is a useful contrast: it curates prior transcripts into persistent context [6], whereas this study changes retained code, tests and the handover. Memory updates can carry an incorrect lesson forward just as code updates can retain a defect. A self-improvement label therefore cannot replace the retention and adaptation measures. The study asks what remains after a fixed repository-consolidation step, with CR1 as one future-change probe. It does not test choosing when to interrupt delivery, changing product goals or restoring a deployment. Those questions, and matched human/agent steering, belong to the separate research agenda in the ancillary supplement.

8.5 Implications for practice

The case motivates protecting a usable increment, bounding the delivery horizon and keeping lessons retrievable or executable. The ancillary supplement retains the proposed practice checklist and framework-builder suggestions. For future product integrations, routine intermediate tracking files should stay out of the squash-merged product diff once durable findings and original review evidence have been preserved. This is a proposed practice, motivated by 478,554 tracking lines added across the two periods, not a tested remedy. It is not permission to wipe an active or historical tracking tree: selection must be finite, archive-backed and restore-verified, with adverse reviews retained. Proportionality never overrides mandatory safety, privacy, approval or publication gates, and compact context views must retain their source evidence. The pilot does not test that practice package, an optimal human-intervention cadence or the preservation of expertise. Those remain separate questions.

9 Conclusion

The unit of progress in cumulative development is the system left for the next change. Knight Capital shows how a new deployment can activate an old fault. WyCash makes consolidation part of continued growth. Agent research adds a direct cross-feature test: in SWE-Maintain, earlier changes affect later tasks even when conversation history is discarded. Its clean-history results make preserving behaviour a more precise target than treating all agent-written code as structurally deficient.

The instructions governing development need maintenance too. Repository studies show repeated additions and revisions, alongside pruning; controlled context-file comparisons show that more guidance can increase work without improving task resolution. Code, tests and instructions are all inherited state, but they require different measures of correctness and burden.

The exploratory testbed supplies a traceable delivery failure within this wider problem: 183 merged pull requests did not restore the owner's usable replacement within the original observation window. The case's five candidate mechanisms frame possible links among accumulated obligations, lost lessons, completion proxies and insufficient product-use feedback. Migration and deployment dependencies remain central to explaining that particular failure. The separately dated repository follow-up records 262 merged pull requests since 15 October but no fresh deployed-journey acceptance (Section 4.6).

The prospective five-arm study asks what scheduled consolidation leaves for later work. It separates final behavioural coverage from retained-contract loss, probes an unforeseen change and reports the resources consumed. Secondary metrics provide descriptive evidence; testing the proposed mechanisms would require further work. The study has not started; a new freeze, prospective OSF recording and collection gates remain pending. The cost-of-inaction scenario exposes break-even assumptions, not observed savings. What matters is whether the agents' combined work preserves the behaviour and options needed by subsequent features.

Acknowledgements

The author thanks the colleagues and partners whose questions shaped this report.

Use of generative AI

GitHub Copilot with HVE Core agents supported development of the system under study. The case method distinguishes reported interface labels from recorded selectors; the deployed assistant exercise used `gpt-4.1-mini-2025-04-14`, not the coding-agent model. Microsoft 365 Copilot supported summarisation and reference discovery.

GitHub Copilot in VS Code assisted log analysis, mechanism naming and organisation, manuscript drafting and revision, bibliographic checks, public repository inspection, figure preparation and local validation. Scientific Agent Skills supplied developmental writing, evaluation and hypothesis guidance [52]. This citation records assistance tooling, not evidence for the case findings or consolidation hypothesis. AI assistance supplied neither an independent empirical coder nor replicated application or study results. The package README retains the revision-by-revision account and source-access limits. GitHub Copilot CLI assisted the 29 October source integration and scoped method-clarity revision. For the subsequent editorial and visual revision, Gemini 4 Argon proposed infographic concepts. Those suggestions were checked against the manuscript and adapted into editable Excalidraw conceptual diagrams; they are not experimental outputs or independent validation. The earlier 1 November v8 revision reconciled the experiment description with v4, added the measurement and conditional cost-of-inaction explanation, and used Gemini 4 Argon for editable infographic design and implementation. AI-generated conceptual graphics and assumed economics are explicitly separate from experimental data; human author approval remains required. The later v9 revision used Copilot-assisted source comparison, static measurements, five-arm kit implementation and local native diagram authoring. It added no experimental trajectories or independent human peer review, and made no new Gemini design-session claim.

The author supplied the observations, selected the case and made the analytical judgements, and reports checking the repository figures and Azure expenditure and confirming the Beck paraphrase after reading the essay and watching the talk. The supplied automatic critique and this revision's focused source and arithmetic checks are assisted evidence assessments, not independent human verification. The package records their coverage and unavailable sources. Metadata checks do not establish claim support; source comparison does not independently validate a source's experiments. The author takes responsibility for the content. Further disclosure must follow the actual submission venue's policy, which has not been assessed here.

Competing interests

The author is a Microsoft employee studying Microsoft and GitHub tools within a Microsoft training programme and designed the coordinator under study. This professional and participant-observer interest is disclosed; it does not establish an independent evaluation or company endorsement. External-publication clearance remains a separate author responsibility.

Data availability

The repository analysed was publicly accessible during the case at https://github.com/jmservera/ama; this report uses commit `29c6b512f5e68733bb2cc1882343db39f5eb2cad`. The separate 29 October repository follow-up pins `9b74ac8daaec2a880172e43161b37ac7f2c3f801`; its selected API records, retrieval interval, merge-window definitions, document hashes and metrics are in anc/repository-metrics-2026-10-29.json. The log-measurement and figure-generation scripts are provided with the source. The ancillary file anc/case-evidence-matrix.json indexes O1–O5, including reported locators and missing release/execution links; it is not newly collected corroboration. The companion anc/supplementary-context.md retains practitioner material, external observations, proposed practices and deferred extensions. The ancillary file anc/repository-observations-2026-10-25.json records the additional repositories' source revisions, tracker counts, sampling boundary and claim locators without reporter identities or attachments. Live issue states can change independently of the pinned source commits. The ancillary experiment notes in HVE Core's Minimum Viable Experiment format are earlier proposals, not the tagged study record. The separate researcher kit is at https://github.com/jmservera/ama-thought-mve; use its historical frozen v4 release. The revised design requires a new exact-source packet and prospective OSF amendment before collection; v4 does not approve it. The author plans to make both repositories private before the experiment, give the independent researcher a protected kit without access to the original repositories or manuscript, and release the approved kit and experiment materials with the paper after collection. OSF registration and embargo are distinct from GitHub visibility. Prior public exposure cannot be undone, and privacy alone cannot prove absence of model contamination. On 1 November the owner explicitly authorised development PR updates while both repositories were public. That interim exposure is separate from the planned post-collection release and must be disclosed to the researcher. Private paid diagnostic code, results and transcripts and runtime image bytes are excluded from that packet. The author-reported original OSF registration does not establish prospective v4 registration. Sanitized pilot data and exact execution provenance still require a later authorized deposit; no completed independent replication is claimed.

References

[1] S. Agarwal, A. Singhal, A. Macvean, B. Vasilescu, and T. Breaux. Spec-driven development as prescribed and as practiced. arXiv:2608.25202, 2026.

[2] Alberto Bacchelli and Christian Bird. Expectations, outcomes, and challenges of modern code review. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, pages 712–721. IEEE, 2013. doi:10.1109/ICSE.2013.6606617.

[3] L. Bainbridge. Ironies of automation. *Automatica*, 19(6):775–779, 1983. doi:10.1016/0005-1098(83)90046-8.

[4] S. Baltes et al. Guidelines for empirical studies in software engineering involving large language models. *Empirical Software Engineering*, in press; arXiv:2508.15503v8, 2026.

[5] G. Bavota, A. De Lucia, M. Di Penta, R. Oliveto, and F. Palomba. An experimental investigation on the innate relationship between quality and refactoring. *Journal of Systems and Software*, 107:1–14, 2015. doi:10.1016/j.jss.2015.05.024.

[6] beamnxw (uploader). 99% Use Agents Wrong: Anthropic's Secret: Self-Improving Agents (FREE GUIDE). Third-party YouTube upload, 18 October 2026; automatic English captions inspected 1 November 2026. The speaker identifies herself as Lamis, Anthropic Applied AI. Original recording date and audio wording unverified; headline percentages are not findings. https://www.youtube.com/watch?v=0CG2YlJZWVU.

[7] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R. C. Martin, S. Mellor, K. Schwaber, J. Sutherland, and D. Thomas. Manifesto for agile software development, 2001. https://agilemanifesto.org/. Accessed 1 November 2026; manifesto excerpt.

[8] K. Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2002.

[9] K. Beck and C. Andres. *Extreme Programming Explained: Embrace Change*. Addison-Wesley, 2nd edition, 2004.

[10] K. Beck. `test && commit || revert`. Medium, 28 September 2018. https://medium.com/@kentbeck_7670/test-commit-revert-870bbd756864. Retrieval attempted 1 November 2026; HTTP 403, full text unavailable.

[11] K. Beck. *Tidy First? A Personal Exercise in Empirical Software Design*. O'Reilly Media, 2023.

[12] K. Beck. Augmented coding: Beyond the vibes. *Software Design: Tidy First?* (newsletter), 25 June 2025. https://tidyfirst.substack.com/p/augmented-coding-beyond-the-vibes. Accessed 1 November 2026; brief excerpt.

[13] K. Beck. Why does development slow? *Software Design: Tidy First?*, 19 November 2025. Author's summary at https://kentbeck.com/summaries/why-does-development-slow/. Accessed 1 November 2026; author's complete summary, not the full essay.

[14] K. Beck, interviewed by G. Orosz. TDD, AI agents and coding with Kent Beck. *The Pragmatic Engineer* podcast, 11 June 2025. https://www.youtube.com/watch?v=aSXaxOdVtAQ. Video-page retrieval attempted 1 November 2026; HTML only, video/transcript not inspected.

[15] K. Beck. Mathematicians, here's a way to think about your existential crisis. *Software Design: Tidy First?* (newsletter), 29 September 2026. https://newsletter.kentbeck.com/p/mathematicians-heres-a-way-to-think. Accessed 1 November 2026; brief excerpt.

[16] K. Beck. Software engineering in the age of AI. Talk at Prodacity 2026, Nashville, TN, 25–27 August 2026. Video: https://www.youtube.com/watch?v=F8fBgDCf2Y4; transcript: https://www.rise8.us/videos/kent-beck-software-engineering-in-the-age-of-ai. Transcript accessed 1 November 2026; video-page HTML only, video not inspected.

[17] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. Measuring the impact of early-2025 AI on experienced open-source developer productivity, 2025.

[18] T. Besker, A. Martini, and J. Bosch. Technical debt cripples software developer productivity: A longitudinal study on developers' daily software development work. In *Proceedings of the International Conference on Technical Debt (TechDebt '18)*, pages 105–114. ACM, 2018. doi:10.1145/3194164.3194178.

[19] C. Bird, V.-P. Ranganath, T. Zimmermann, N. Nagappan, and A. Zeller. Extrinsic influence factors in software reliability: A study of 200,000 Windows machines. In *ICSE Companion*, pages 205–214. ACM, 2014. https://www.microsoft.com/en-us/research/publication/extrinsic-influence-factors-in-software-reliability-a-study-of-200000-windows-machines/. Accessed 1 November 2026; publisher abstract and ACM/Crossref metadata.

[20] B. Böckeler. Understanding spec-driven-development: Kiro, spec-kit, and Tessl. martinfowler.com, 15 October 2025. https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html. Accessed 1 November 2026; brief excerpt.

[21] M. Borg, D. Hewett, N. Hagatulah, N. Couderc, E. Söderberg, D. Graham, U. Kini, and D. Farley. Echoes of AI: Investigating the downstream effects of AI assistants on software maintainability. *Empirical Software Engineering*, 31(6):161, 2026. doi:10.1007/s10664-026-10889-1.

[22] F. P. Brooks, Jr. No silver bullet: Essence and accidents of software engineering. *Computer*, 20(4):10–19, 1987. doi:10.1109/MC.1987.1663532.

[23] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025), Datasets and Benchmarks Track*, 2025.

[24] W. Chatlatanagulchai, H. Li, Y. Kashiwa, B. Reid, K. Thonglek, P. Leelaprute, A. Rungsawang, B. Manaskasemsak, B. Adams, A. E. Hassan, and H. Iida. Agent READMEs: An empirical study of context files for agentic coding. *ACM Transactions on Software Engineering and Methodology*, 2026. doi:10.1145/3840295. Inspected text: arXiv:2511.12884v2. https://arxiv.org/abs/2511.12884v2.

[25] L. Chen, M. Zaharia, and J. Zou. How is ChatGPT's behavior changing over time? arXiv:2307.09009v3, 2023. https://arxiv.org/abs/2307.09009v3.

[26] J. Chen, X. Xu, H. Wei, C. Chen, and B. Zhao. SWE-CI: Evaluating agent capabilities in maintaining codebases via continuous integration. arXiv:2603.03823, 2026.

[27] A. Cockburn. *Crystal Clear: A Human-Powered Methodology for Small Teams*. Addison-Wesley, 2004.

[28] J. Cohen. *Best Kept Secrets of Peer Code Review*. Smart Bear Inc., 2006.

[29] W. Cunningham. The WyCash portfolio management system. In *Addendum to the Proceedings of OOPSLA '92 (Experience Report)*, pages 29–30. ACM, 1992. doi:10.1145/157709.157715.

[30] D. Delimarsky. Spec-driven development with AI: Get started with a new open source toolkit. The GitHub Blog, 2 September 2025. https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/. Accessed 1 November 2026; brief excerpt.

[31] P. Devanbu, T. Zimmermann, and C. Bird. Belief & evidence in empirical software engineering. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. ACM, 2016. doi:10.1145/2884781.2884812.

[32] Jingzhe Ding, Shengda Long, Changxin Pu, et al. NL2Repo-Bench: Towards long-horizon repository generation evaluation of coding agents, 2025.

[33] DORA. State of AI-assisted software development 2025. Technical report, Google Cloud, 2025. URL https://dora.dev/dora-report-2025. Accessed 1 November 2026; landing-page metadata, not the report's full text.

[34] S. G. Eick, T. L. Graves, A. F. Karr, J. S. Marron, and A. Mockus. Does code decay? Assessing the evidence from change management data. *IEEE Transactions on Software Engineering*, 27(1):1–12, 2001. doi:10.1109/32.895984.

[35] S. M. Eldridge, C. L. Chan, M. J. Campbell, C. M. Bond, S. Hopewell, L. Thabane, and G. A. Lancaster. CONSORT 2010 statement: extension to randomised pilot and feasibility trials. *BMJ*, 355:i5239, 2016. doi:10.1136/bmj.i5239. Accessed 1 November 2026; methods/reporting excerpt.

[36] M. Fowler. Design stamina hypothesis. martinfowler.com, 20 June 2007. https://martinfowler.com/bliki/DesignStaminaHypothesis.html. Accessed 1 November 2026; full web essay.

[37] M. Fowler. Some thoughts on LLMs and software development. martinfowler.com, 28 August 2025. https://martinfowler.com/articles/202508-ai-thoughts.html. Accessed 1 November 2026; brief excerpt.

[38] J. Gall. *General Systemantics: An Essay on How Systems Work, and Especially How They Fail*. General Systemantics Press, Ann Arbor, MI, 1975.

[39] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995. ISBN 978-0-201-63361-0. Publisher's preface and Adapter excerpt: https://www.informit.com/store/design-patterns-elements-of-reusable-object-oriented-9780201633610; https://www.informit.com/articles/article.aspx?p=1398600. Accessed 1 November 2026; publisher description, not the full book.

[40] D. Gautam, S. Garg, J. Jang, N. Sundaresan, and R. Zilouchian Moghaddam. RefactorBench: Evaluating stateful reasoning in language agents through code. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2503.07832.

[41] T. Gloaguen, N. Mündler-Sasahara, M. N. Müller, V. Raychev, and M. Vechev. Evaluating AGENTS.md: Are repository-level context files helpful for coding agents? arXiv:2602.11988v3, 29 September 2026. https://arxiv.org/abs/2602.11988v3.

[42] Eliyahu M. Goldratt and Jeff Cox. *The Goal: A Process of Ongoing Improvement*. North River Press, 1984.

[43] C. A. E. Goodhart. Problems of monetary management: The U.K. experience. In *Papers in Monetary Economics*, volume I, pages 1–20. Reserve Bank of Australia, Sydney, 1975.

[44] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *ICLR*, 2024. Inspected text: arXiv:2305.11738v4, Section 3; accessed 1 November 2026. https://arxiv.org/abs/2305.11738v4.

[45] A. Herdağdelen, H. Joren, and A. Tamkin. How AI agents shape experts' judgment calls. Anthropic research article, 27 October 2026; accessed 29 October 2026. https://www.anthropic.com/research/agents-judgment-calls. Accessed 1 November 2026; study design, results, limitations and proposed safeguards.

[46] I. Herraiz, D. Rodriguez, G. Robles, and J. M. Gonzalez-Barahona. The evolution of the laws of software evolution: A discussion based on a systematic literature review. *ACM Computing Surveys*, 46(2):28, 2013. doi:10.1145/2543581.2543595.

[47] Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts LLM performance. Technical report, Chroma, 2025. URL https://trychroma.com/research/context-rot. Accessed 1 November 2026; brief excerpt.

[48] Ryan Hoque, Ashwin Balakrishna, Ellen Novoseller, Albert Wilcox, Daniel S. Brown, and Ken Goldberg. ThriftyDAgger: Budget-aware novelty and risk gating for interactive imitation learning. In *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 598–608. PMLR, 2022. URL https://proceedings.mlr.press/v164/hoque22a.html.

[49] A. Hunt and D. Thomas. *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley, 1999.

[50] Y. Jiang, K. C. Black, G. Geng, D. Park, J. Zou, A. Y. Ng, and J. H. Chen. MedAgentBench: A virtual EHR environment to benchmark medical LLM agents. *NEJM AI*, 2(9):AIdbp2500144, 2025. doi:10.1056/AIdbp2500144. Inspected text: arXiv:2501.14654v2; accessed 1 November 2026. Publisher full text unavailable.

[51] Z. Jiang, F. Huang, H. Xing, X. Wu, Y. Gao, R. Cao, M. Wang, S. Liu, and Y. Li. Demystifying agent skills: Why they work–until they don't. arXiv preprint arXiv:2608.14036v1, 2026; accessed 29 October 2026. https://arxiv.org/html/2608.14036v1.

[52] T. Kassis, V. Agarwal, Y. He, D. Patel, and A. M. Brueckner. Scientific agent skills: A library of procedural knowledge for research agents. arXiv:2609.00065, 2026. doi:10.48550/arXiv.2609.00065.

[53] Michael Kelly, Chelsea Sidrane, Katherine Driggs-Campbell, and Mykel J. Kochenderfer. HG-DAgger: Interactive imitation learning with human experts. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 8077–8083, 2019. First posted 2018.

[54] M. Kim, T. Zimmermann, and N. Nagappan. An empirical study of refactoring challenges and benefits at Microsoft. *IEEE Transactions on Software Engineering*, 40(7):633–649, 2014. doi:10.1109/TSE.2014.2318734.

[55] A. Kumar, Y. Bajpai, S. Gulwani, G. Soares, and E. Murphy-Hill. Why AI agents still need you: Findings from developer-agent collaborations in the wild. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 432–444, 2025. arXiv:2506.12347v3, 9 October 2025.

[56] P. Laban, H. Hayashi, Y. Zhou, and J. Neville. LLMs get lost in multi-turn conversation. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2505.06120v1, 9 May 2025.

[57] H.-P. Lee, A. Sarkar, L. Tankelevitch, I. Drosos, S. Rintel, R. Banks, and N. Wilson. The impact of generative AI on critical thinking: Self-reported reductions in cognitive effort and confidence effects from a survey of knowledge workers. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pages 1–22. ACM, 2025. doi:10.1145/3706598.3713778.

[58] M. M. Lehman. Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9):1060–1076, 1980. doi:10.1109/PROC.1980.11805.

[59] M. M. Lehman, J. F. Ramil, P. D. Wernick, D. E. Perry, and W. M. Turski. Metrics and laws of software evolution: The nineties view. In *Proceedings of the 4th International Software Metrics Symposium (METRICS '97)*. IEEE, 1997. doi:10.1109/METRIC.1997.637156.

[60] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. AIDev: Studying AI coding agents on GitHub. In *Proceedings of the International Conference on Mining Software Repositories (MSR)*, 2026. doi:10.1145/3793302.3797249.

[61] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. https://doi.org/10.1162/tacl_a_00638.

[62] R. Lopopolo. Harness engineering: Leveraging Codex in an agent-first world. OpenAI, 11 February 2026. https://openai.com/index/harness-engineering/. Retrieval attempted 1 November 2026; HTTP 403, fresh full text unavailable.

[63] J. L. Lulla, M. Galster, J. M. Zhang, S. Baltes, and C. Treude. The impact of configuring agentic AI coding tools on build-vs-buy decisions: A study protocol. ESEM 2026, Registered Reports track. arXiv:2606.03907, 2026.

[64] Y. Ma, B. Kereopa-Yorke, and B. Schultz. Building to the test: Coding agents deliver what you check, not what you requested. arXiv preprint arXiv:2606.28430v1, 26 June 2026. https://arxiv.org/abs/2606.28430v1.

[65] A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. arXiv:2303.17651v2, 2023; Section 2 inspected 1 November 2026. https://arxiv.org/abs/2303.17651v2.

[66] R. C. Martin. Why hardcore technical focus still matters big time. *Product Engineers* podcast, 29 September 2026. https://productengineers.com/episodes/season2/why-hardcore-technical-focus-still-matters-big-time-uncle-bob/. Accessed 1 November 2026; episode description, not the complete audio.

[67] S. Mhatre, Y. Bajpai, S. Gulwani, E. Murphy-Hill, and G. Soares. SWE-Sharp-Bench: A reproducible benchmark for C# software engineering tasks. In *International Conference on AI-powered Software (AIware)*, 2025. Accessed arXiv:2511.02352v2, 5 November 2025.

[68] Microsoft Research. Systematic debugging for AI agents: Introducing the AgentRx framework. First-party research blog, 12 March 2026; accessed 27 October 2026. https://www.microsoft.com/en-us/research/blog/systematic-debugging-for-ai-agents-introducing-the-agentrx-framework/. Accessed 1 November 2026; first-party summary excerpt.

[69] Microsoft. HVE Core: Hypervelocity Engineering Core. GitHub repository and documentation, tag `plugins-v3.3.106`, accessed 24 October 2026. https://github.com/microsoft/hve-core/tree/plugins-v3.3.106; https://microsoft.github.io/hve-core/. Tag recheck attempted 1 November 2026; organization SAML denied access, no bypass attempted.

[70] Microsoft Research. SkillOpt: Agent skills as trainable parameters. First-party research blog, 30 June 2026; accessed 27 October 2026. https://www.microsoft.com/en-us/research/blog/skillopt-agent-skills-as-trainable-parameters/. Accessed 1 November 2026; first-party summary excerpt.

[71] S. Mohsenimofidi, M. Galster, C. Treude, and S. Baltes. Context engineering for AI agents in open-source software. In *MSR '26*, pages 194–198, 2026. doi:10.1145/3793302.3793350. Inspected text: arXiv:2510.21413v4. https://arxiv.org/abs/2510.21413v4.

[72] Andrew Murphy. If you thought the speed of writing code was your problem – you have bigger problems. Blog post, 17 March 2026, https://andrewmurphy.io/blog/if-you-thought-the-speed-of-writing-code-was-your-problem-you-have-bigger-problems, 2026. Accessed 1 November 2026; page metadata/brief excerpt.

[73] Emerson Murphy-Hill, Jenna Butler, and Alexandra Savelieva. Adoption and impact of command-line AI coding agents: A study of Microsoft's early 2026 rollout of Claude Code and GitHub Copilot CLI, 2026.

[74] P. Naur. Programming as theory building. *Microprocessing and Microprogramming*, 15(5):253–261, 1985. doi:10.1016/0165-6074(85)90032-8.

[75] A. Ng. Agentic design patterns part 2: Reflection. *The Batch*, DeepLearning.AI, 27 March 2024; accessed 1 November 2026. Practitioner letter, not a controlled study. https://www.deeplearning.ai/the-batch/agentic-design-patterns-part-2-reflection/.

[76] M. Nigh. How to write a great agents.md: Lessons from over 2,500 repositories. The GitHub Blog, 19 November 2025, updated 25 November 2025. https://github.blog/ai-and-ml/github-copilot/how-to-write-a-great-agents-md-lessons-from-over-2500-repositories/. Accessed 1 November 2026; brief excerpt.

[77] G. Orlanski, D. Roy, A. Yun, C. Shin, A. Gu, A. Ge, D. Adila, N. Roberts, F. Sala, and A. Albarghouthi. SlopCodeBench: Benchmarking how coding agents degrade over long-horizon iterative tasks. arXiv:2603.24755v2, 2026. Conference status requires separate verification; this citation pins the inspected preprint version.

[78] R. Pan, J. Zhang, L. Wu, Y. Zhang, H. Zhang, and L. Zhang. Do modifications made by coding agents make subsequent software maintenance harder? arXiv preprint arXiv:2610.14768v1, 10 October 2026.

[79] S. Park, W. Kim, R. Tan, J. Zhang, W.-S. Han, P. Gao, C. Park, Y. Yao, R. Fu, E. Nallipogu, Q. Lin, S. Rajmohan, and D. Zhang. AutoSaddler: Automatic harness optimization with durable updates from agent execution traces. arXiv:2608.23041v1, 24 August 2026; abstract inspected 1 November 2026. https://arxiv.org/abs/2608.23041v1.

[80] D. L. Parnas. Software aging. In *Proceedings of the 16th International Conference on Software Engineering (ICSE '94)*, pages 279–287. IEEE, 1994. doi:10.1109/ICSE.1994.296790.

[81] A. Pavlenko, B. Crivat, B. Haynes, C. Curino, F. Psallidas, J. Slawinski, J. Freischuetz, L. Pereira Sanchez, M. Weimer, M. Demarne, M. Jasny, M. Gandhi, M. Bovykin, M. Milletari, P. Ghosh, Q. Bai, R. Ramakrishnan, R. Pandita, S. Matusevich, S. Venkataraman, S. Krishnan, M. T. Mahmood, T. Bang, V. Emani, X. Zhao, and Y. Zhu. Towards an AI software factory for data systems. arXiv:2609.36323v1, 28 September 2026. https://arxiv.org/abs/2609.36323v1.

[82] Sien Reeve O. Peralta, Fumika Hoshi, Hironori Washizaki, Naoyasu Ubayashi, Inase Kondo, Yoshiki Higo, Hiroki Mukai, Norihiro Yoshida, Kazuki Kusama, Hidetake Tanaka, and Youmei Fan. Why are agentic pull requests merged or rejected? an empirical study. In *Proceedings of the 23rd International Conference on Mining Software Repositories*, pages 787–791, 2026. doi:10.1145/3793302.3793575.

[83] philia (uploader). Prompting Is Dead in 6 Months. Andrew Ng, Stanford. Third-party YouTube upload, 4 September 2026; automatic English captions inspected 1 November 2026. Ng introduces Laurence Moroney at 16:31, who gives the later technical-debt and reflection discussion. No caption support for the headline's six-month prediction. https://www.youtube.com/watch?v=9EuNUe-CJRM.

[84] PhotoCraft contributors. PhotoCraft: Source, parity records and public tracker observations. GitHub, source commit `5bb8536`; retrieved 25 October 2026. https://github.com/storytold/photocraft/tree/5bb853654196eca807212ff8955ba2eb293071ea. Dated counts and claim locators in the ancillary repository-observation record. Commit metadata accessed 1 November 2026 through GitHub CLI; application not executed.

[85] P. Rajasekaran, E. Dixon, C. Ryan, and J. Hadfield. Effective context engineering for AI agents. Anthropic, 29 September 2025. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents. Accessed 1 November 2026; brief excerpt at redirected official page.

[86] P. Ralph et al. Empirical standards for software engineering research. arXiv:2010.03525v2, 2021 (first posted 2020). ACM SIGSOFT Empirical Standards: https://www2.sigsoft.org/EmpiricalStandards/. Accessed 1 November 2026; standards landing-page excerpt.

[87] W. W. Royce. Managing the development of large software systems. In *Proceedings of IEEE WESCON*, pages 1–9, August 1970.

[88] P. Runeson and M. Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164, 2009. doi:10.1007/s10664-008-9102-8.

[89] J. M. Servera Bondroit. AMA Case Study 48: research and solution design for the Zava Forward retrofit programme. GitHub repository, commit 29c6b51, 24 October 2026. https://github.com/jmservera/ama. Repository/commit and named issue/PR metadata accessed 1 November 2026; historical live journey unverified.

[90] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023. Inspected text: arXiv:2303.11366v4, Section 3; accessed 1 November 2026.

[91] A. Simkute, L. Tankelevitch, V. Kewenig, A. E. Scott, A. Sellen, and S. Rintel. Ironies of generative AI: Understanding and mitigating productivity loss in human-AI interaction. *International Journal of Human–Computer Interaction*, 41(5):2898–2919, 2025. doi:10.1080/10447318.2024.2405782.

[92] A. Sinha, A. Arun, S. Goel, S. Staab, and J. Geiping. The illusion of diminishing returns: Measuring long horizon execution in LLMs. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2509.09677v3, 13 March 2026.

[93] M. Strathern. 'Improving ratings': Audit in the British university system. *European Review*, 5(3):305–321, 1997.

[94] L. Tankelevitch, V. Kewenig, A. Simkute, A. E. Scott, A. Sarkar, A. Sellen, and S. Rintel. The metacognitive demands and opportunities of generative AI. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, pp. 1–24. ACM, 2024. doi:10.1145/3613904.3642902.

[95] A. Tornhill and M. Borg. Code red: The business impact of code quality. A quantitative study of 39 proprietary production codebases. In *Proceedings of the International Conference on Technical Debt (TechDebt '22)*, pages 11–20. ACM, 2022. doi:10.1145/3524843.3528091.

[96] U.S. Securities and Exchange Commission. In the matter of Knight Capital Americas LLC. Order, Exchange Act Release 34-70694, 16 October 2013. https://www.sec.gov/files/litigation/admin/2013/34-70694.pdf. Order retrieval attempted 1 November 2026; HTTP 403, fresh full text unavailable.

[97] M. Waseem, A. Ahmad, K.-K. Kemell, J. Rasku, S. Lahti, K. Mäkelä, and P. Abrahamsson. Vibe coding in practice: Flow, technical debt, and guidelines for sustainable use. arXiv:2512.11922, 2025.

[98] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén. *Experimentation in Software Engineering*. Springer, 2012. doi:10.1007/978-3-642-29044-2.

[99] Walden Yan. Don't build multi-agents. Cognition blog, 12 June 2025, https://cognition.com/blog/dont-build-multi-agents, 2025. Accessed 1 November 2026; context-sharing principles within the argument against multi-agent designs.

[100] C. Yang, H. He, T.-C. Cheng, J. Butler, E. Murphy-Hill, C. Badea, and C. Bird. (When) should a software factory be lights-off? Benchmarking code review agents on bug detection. arXiv preprint arXiv:2610.13668v1, 9 October 2026. https://arxiv.org/abs/2610.13668v1.

[101] J. Young. Effective harnesses for long-running agents. Anthropic, 26 November 2025. https://www.anthropic.com/research/effective-harnesses-for-long-running-agents. Accessed 1 November 2026; brief excerpt, not a fresh complete study audit.

[102] H. Yu, L. Liu, X. Jiang, Y. Jia, S. Wang, P. Qian, and Y. Chen. Habituation at the gate: Rising approval and declining scrutiny in human review of AI agent code. KDD 2026 Workshop on Agentic Software Engineering, 2026. arXiv:2606.22721.

[103] M. Zhang, O. Press, W. Merrill, A. Liu, and N. A. Smith. How language model hallucinations can snowball. In *ICML*, PMLR 235:59670–59684, 2024. Inspected original text: arXiv:2305.13534v1; https://proceedings.mlr.press/v235/zhang24ay.html.

[104] Z. Zhong, A. Raghunathan, and N. Carlini. ImpossibleBench: Measuring LLMs' propensity of exploiting test cases. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.20270.

A Supplement

This section retains the extended background, detailed rival-explanation register, candidate-mechanism audits and historical measurement definitions. It adds no experimental results or independent human corroboration. The five-arm prospective protocol is in Section 7 and the current researcher kit. The consolidation prompt reproduced here is the historical v4 prompt, not the amended experiment's approved execution source. Expanded discussions and repeated published-data figures provide reference detail; they are not additional observations or independent studies.

A.1 Case evidence and lexical indicator

The supplementary case evidence retains the full rival register and five candidate-mechanism audits. Whitespace counts describe text volume, not loaded context. The evidence-boundary lexicon matches `unmet`, `not accepted`, `not inferred`, `is inferred`, `remains open/unproved/unverified`, `not claimed`, `no ... is/was claimed/inferred`, and `not established`, case-insensitively. Density is matches per 1,000 whitespace-delimited entry words, not a fraction of statements or a general hedging measure. The day-one evidence-based ticking rule changed reporting conventions; it is a rival explanation for the lexical increase. The matched sample and qualified precision assessment are retained with revision metrics. Appropriate caution is not evidence of failure or honesty.

Deferred intervention definitions and historical freeze hashes are in the supplementary appendices, not collected outcomes.

A.2 Background

A.2.1 Software that only grows

That software decays when it only accumulates features is one of the oldest observations in software engineering. Lehman's laws of software evolution hold that a program in real-world use must keep changing, and that its complexity grows as it evolves unless work is done to maintain or reduce it [58]; a later formulation adds that its quality will appear to decline unless it is rigorously maintained and adapted [59]. Parnas called the result software aging, and blamed in part the changes made by people who do not understand the original design [80]. Eick and colleagues found mixed but, on the whole, persuasive evidence of code decay in fifteen-plus years of change data from a large telephone-switching system [34]. Cunningham's technical-debt metaphor framed the cost of not consolidating as interest paid on every later change [29], and Fowler's design stamina hypothesis holds that without attention to design, adding features eventually becomes slower than it would have been with it [36]. Studies have since put numbers on the interest: across 39 proprietary code bases, low-quality files had fifteen times more defects and their issues took 124% longer to resolve in a CodeScene-associated vendor study [95]; 43 developers self-reported losing about 23% of their working time to technical debt in a longitudinal study [18]. These are different outcomes and populations.

The evidence is not universal. Increasing complexity and declining quality are among the less strongly supported of Lehman's laws, and some systems have grown without the predicted decay [46]. Several findings above are observational, and their estimates belong to the studied settings, not every code base.

The engineering intuition often called *software entropy* is that keeping a changing system coherent requires counter-work: removing obsolete paths, restoring boundaries and revisiting assumptions. Software does not deteriorate simply because time passes, and thermodynamic entropy supplies no measure of its quality. Here the measurable quantities are losses of required behaviour, excess structural burden and the effort needed for a later change.

A.2.2 What carries from one feature to the next?

Feature accumulation is the addition of capabilities and interactions; churn is the editing activity that implements, replaces or removes them. Neither is itself deterioration. A *regression* loses previously working behaviour that is still required. A never-implemented feature is instead an acquisition failure. Persistence means that the same fault remains unresolved; propagation means that a later, distinct fault or failure can be traced to reuse of the earlier one. *Compounding* is the stronger claim that the earlier error increases subsequent error risk or change burden.

These distinctions matter because an agent inherits more than a code diff. The repository supplies implementations, tests, interfaces and deployment assumptions. Instructions, skills and notes supply an account of how to work on them. A mistaken rule can become code, then a test expectation or an instruction, and finally the precedent used for another feature. The stored record need not be correct to become authoritative in the next session. Conversely, an inherited implementation can be sound even when it differs from a human reference. The question is what subsequent work can achieve from that state.

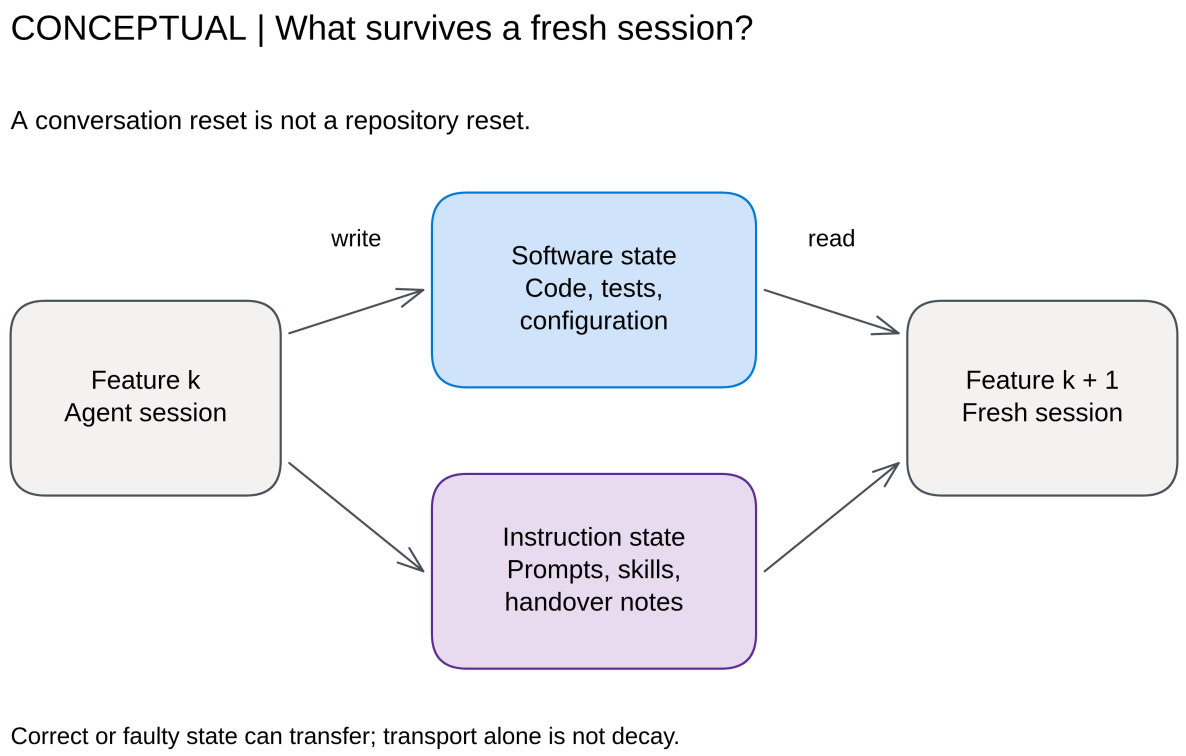


Figure 10: Conceptual state transfer between feature sessions. A fresh conversation does not reset inherited software or instruction state. Solid arrows denote possible read/write paths, not measured error propagation. Both channels can carry correct decisions as well as faults; their presence alone is not evidence of decay.

Figure 10 separates the two persistent channels from conversation history. It is an explanatory model, not an observed trajectory or a claim that every tool retains the same material.

A.2.3 Compounding within and between features

Errors carried in conversation. An earlier answer changes the context from which a model produces the next one. Experiments on this pathway distinguish dependence on an erroneous history from the difficulty of a long task.

Zhang et al.'s earlier *How Language Model Hallucinations Can Snowball* shows why consistency is not enough. On three question-answering datasets, ChatGPT and GPT-4 produce false

justifications after wrong answers, yet often reject the individual false claims when questioned in a separate session. Their explanation is over-commitment to the earlier answer rather than only a lack of knowledge. The inspected original experiments concern question answering in two 2023 models; the later ICML publication expands the model coverage [103, original preprint, Secs. 3.1–3.4].

Sinha et al. test the distinction directly in a controlled execution task. Models receive both the lookup dictionary and the plan, then maintain a running sum. The authors vary the fraction of incorrect answers in artificial prior histories and measure the accuracy of subsequent state updates, not merely whether the final total remains correct. More erroneous history reduces later accuracy beyond the effect of context length. This is counterfactual evidence of *self-conditioning*: earlier mistakes can increase the chance of new mistakes. Scaling non-thinking models does not remove the effect, but Qwen3 models with thinking enabled avoid it in this setup. The result identifies a dependence on model configuration as well as erroneous history [92, Secs. 2.2–3, Figs. 5–6].

Laban et al. supply complementary conversational evidence. Across six generation tasks, including Python and SQL, they report a 39% average relative performance drop when instructions arrive through underspecified multi-turn conversation rather than all at once. A concatenated-instruction control largely preserves performance, weakening information loss from rephrasing as the explanation. Their analyses associate premature answers with lower success and describe reliance on earlier incorrect attempts and increasingly bloated solutions. These are simulated conversations and diagnostic analyses, not a controlled estimate of the cost of a particular inherited coding defect; the 39% is not a percentage-point loss or a repository defect rate [56, arXiv v1, Secs. 3.3 and 6.1, App. F.1–F.2].

Errors carried in the repository. Pan et al.'s *SWE-Maintain* preprint tests the pathway most directly relevant to successive feature work. It reconstructs 100 three-event maintenance episodes from 50 Python repositories and runs each later task from two starting points: the developers' preceding patches and the agent's own preceding changes. Task, agent and tests are held fixed, and every event uses a fresh session: only the repository carries the earlier work forward. Across 1,996 paired later events, agent history lowers new-task resolution by 5.6 percentage points (95% interval $[-7.2, -4.1]$). Requiring earlier tests to remain passing widens the gap to 14.0 points. The unweighted mean new-task gap grows from 4.0 points at the second event to 7.2 at the third, consistent with an accumulating downstream cost, though three-event episodes cannot establish a general long-horizon decay law [78, Secs. 2.5 and 4.1, Table 2].

Their `python_unidiff` example makes the proposed propagation concrete: an agent splits a string into lines, losing line terminators; the next task reuses that logic in a constructor, while developer history succeeds. The framing appears in Figure 1's caption and the detailed failure in Section 5.1 and Figure 5 [78]. Their `parse_crontab` example exposes a less visible dependency: an agent's local Python 3 fix passes the current and next tests, but later code relies on module-wide compatibility aliases that it never supplied. The later compatibility failure is discussed in Section 5.1 and Figure 5 [78]. Among the 619 later events whose inherited code passes earlier tests and whose next task is not already solved, the new-task-only difference is -1.1 points (95% interval $[-3.8, +1.6]$), and the event-resolution difference is $+2.7$ points (95% interval $[-0.5, +6.1]$), with no detected disadvantage. This is a selected clean-history subset, not a randomised repair intervention, and it does not prove equivalent maintainability under all changes. It points more directly to inherited behavioural failures than to structural complexity as the main observed channel [78, Sec. 4.2, Table 3; Sec. 5.1, Fig. 5].

Figure 11 shows the published event-resolution contrasts, including the clean-history cohorts. Event resolution requires the new task and all earlier non-superseded tests to pass. Its larger gap therefore includes retained failures as well as difficulty solving the new task. The current-task-only difference of -5.6 points is important for the cross-feature hypothesis precisely because it

does not count earlier failing tests against success on the new task.

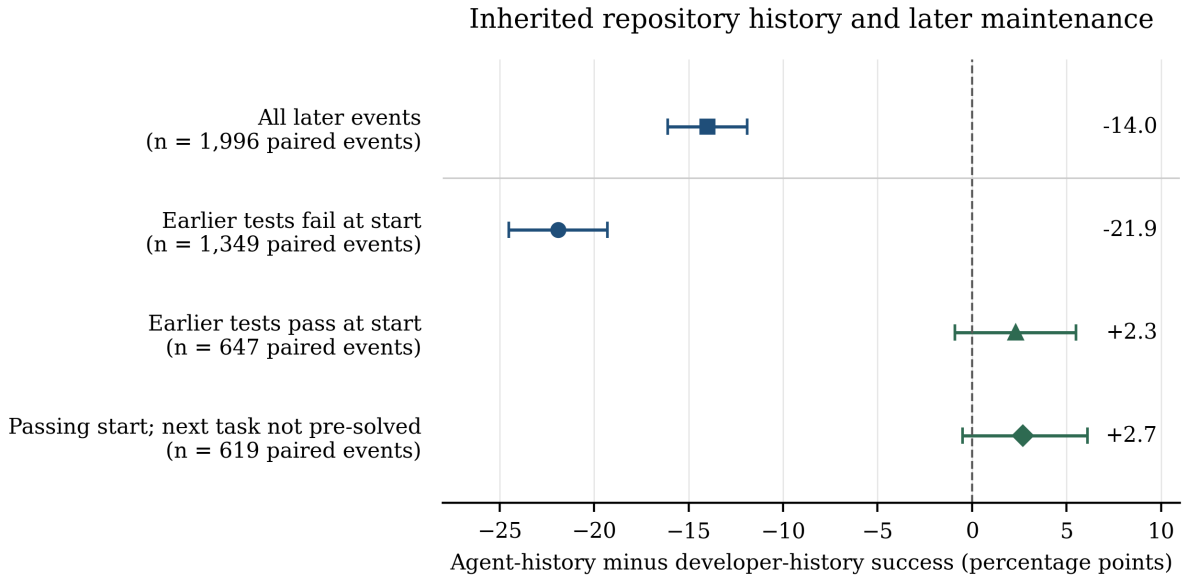


Figure 11: Later maintenance from two repository histories. Points and 95% repository-clustered bootstrap intervals are transcribed from Pan et al., Table 3 [78]. The outcome is event resolution: current and all earlier non-superseded hidden tests pass. Negative differences favour developer history. The 1,996 paired events come from 100 three-event episodes in 50 Python repositories, evaluated with five systems and two models. Subgroups overlap and are selected by the agent-history starting state; the last excludes pre-solved and unevaluable tasks. They are not randomised repair conditions.

The conversation and repository studies therefore locate two different transmission pathways. Resetting context addresses the first, but not faulty code retained for the next feature. SlopCodeBench and SWE-CI extend the evaluation horizon to iterative structural erosion and behavioural regressions [26, 77]. A maintainability study using late-2024 coding assistants found no significant downstream difference; only its second phase was randomised [21]. This is assistant-era evidence, not a test of the current autonomous agents. The emerging question is which inherited failures survive, how they affect later work, and which interventions prevent or repair them.

Output and review capacity. Understanding changes is a major challenge of code review; habituation, fragmented context and uneven retrieval from long contexts may limit scrutiny as output grows [2, 23, 47, 61, 102]. Faster feature delivery also shortens the time available to discover that an earlier assumption has become the foundation of another change. Further empirical and practitioner context is retained in the ancillary supplement.

A.2.4 Corrective instructions as maintenance debt

A failed agent action often invites a new instruction: run this check, never edit that file, remember this exception. GitHub's account of more than 2,500 agent-definition files recommends starting small, testing and adding detail when an agent makes mistakes [76]. The resulting guidance can preserve useful lessons, but it also becomes another maintained system. Chatlatanagulchai et al. document ongoing maintenance of this material [24, Secs. 4.2.2–4.2.3]. Here *instruction debt* denotes the avoidable burden of obsolete, duplicated, conflicting or poorly routed directives in prompts, skills and handover notes. This extends the maintenance question from the product to the machinery that changes it.

Repository histories show that this material evolves. Mohsenimofidi et al. found no post-creation change in 77 of 155 instruction files. In the ten most frequently changed files, however,

they classified 169 commits: 78 added instructions, 59 modified them and 23 removed them. These are three of nine categories, with 242 assignments in total, rather than a partition of the 169 commits [71, Sec. 4.3, Table 2]. Chatlatanagulchai et al.'s manual analysis of 100 post-adoption commits produced 78 addition, 32 modification and 19 deletion labels; addition labels occurred across 66 distinct commits. Figure 12 replots the label counts [24, Secs. 4.2.2–4.2.3]. These histories contain accumulation and pruning, rather than a uniform march towards longer files. The authors associate their day-scale edit intervals with ongoing project maintenance rather than immediate prompt debugging.

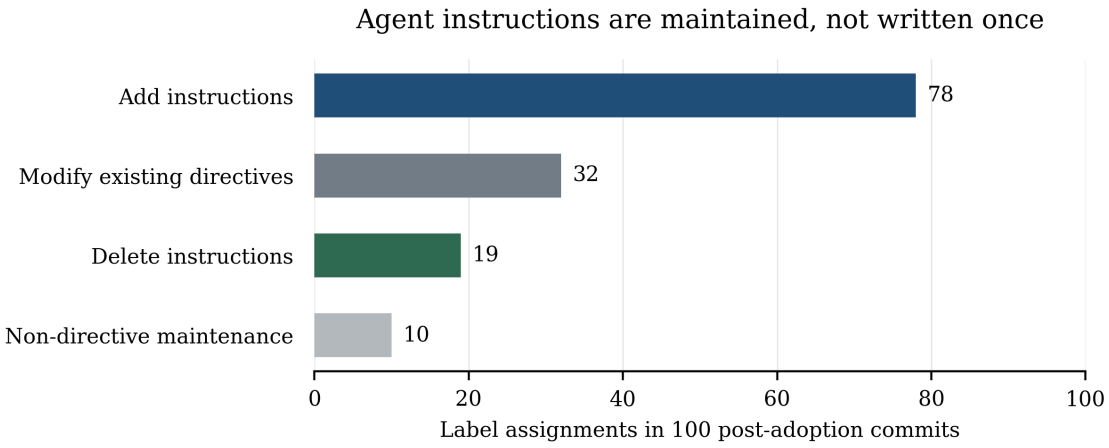


Figure 12: Instruction-file maintenance in 100 randomly sampled post-adoption commits, from Chatlatanagulchai et al., Sections 4.2.2–4.2.3 [24]. Commits can receive several labels; line-level and section-level add/delete labels are combined. The ten non-directive maintenance labels are the remainder of 139 total assignments. The chart counts edit types, not net token growth or effects on agent performance.

Gloaguen et al. test the effect of supplying such context. Across four agent–model pairings on 300 SWE-bench Lite tasks and 138 CTXbench tasks, LLM-generated context files increased average inference cost by 20% and 23%, respectively; task success was slightly lower, by 0.5 and 2 percentage points, without significant differences from the no-file baseline. Developer-written files performed significantly better than LLM-written files on CTXbench ($p = 0.038$). Agents followed the instructions, performed more testing and exploration, and spent more reasoning tokens. Yet file length did not explain success, and generated context helped when other documentation was removed [41, Secs. 4.2–4.4, App. B]. Instruction count and loaded tokens are therefore different exposures from the obligations and behaviours those instructions induce.

Anthropic's context-engineering account describes the practical tension: provide enough information to guide action, but avoid brittle prompt logic and an expanding catalogue of exceptions [85]. Maintaining these instructions means preserving necessary constraints while replacing accumulated exceptions with coherent rules, executable checks and task-specific retrieval. To test the corrective-rule spiral directly would require versioned instructions, actual session inputs and outcomes across successive revisions. The testbed record supplies document growth and rule changes; the consolidation protocol tests a bounded form of handover maintenance by replacing, rather than appending to, its design note.

SkillOpt supplies counterevidence to treating instruction change as inherently harmful. Its first-party account describes bounded edits to skill files, held-out validation and retention of successful versions without changing the target model's weights [70]. Procedural knowledge needs evaluation and maintenance; its length alone tells us little about its quality.

Jiang et al.'s preprint compares workflow memory with skills distilled from the same execution histories. Its paired trajectory analysis associates skills chiefly with more reliable procedures, while also identifying misapplication and insufficient runtime verification [51, Secs. 3.3, 5.1–5.3]. This qualifies an instruction-debt account: retained guidance can stabilise execution rather

than merely burden context. Their retrieval study also separates skill access from task success; in their comparison, lower exact-match access was observed alongside sustained downstream success [51, Sec. 5.5]. Because retrieval diagnostics are run separately from task execution, this comparison does not isolate the effect of reduced skill access. Neither result measures the effects of accumulating repository instructions across feature changes.

A.2.5 Refactoring, review and evaluation boundaries

Agentic refactoring, review and human correction are already studied. RefactorBench evaluates 100 handcrafted multi-file refactoring tasks across nine Python repositories, not scheduled consolidation between feature additions [40]. Kumar et al.'s ASE 2025 study observed developers collaborating with agents on existing issues: incremental, active collaboration was associated with greater success, not randomly assigned to establish an optimal intervention rate [55]. SWE-Sharp-Bench supplies a reproducible C# issue-resolution benchmark and curation pipeline [67]; reproducible repository-scale evaluation itself is not a contribution claimed here.

Two recent preprints sharpen the evaluation problem. CRBugBench evaluates review agents on 1,000 open-source and 800 proprietary tasks from one company, reporting weaker detection on complex, non-local and proprietary-context changes [100]. It is a 9 October 2026 preprint, not an independently replicated result establishing that review cannot work. *Building to the Test* reports near-perfect behavioural scores for demos while the requested reusable library was unused or absent in seven of twelve oracle-in-the-loop runs. Its unchanged, source-hidden oracle did not check that delivery contract [64]. This is distinct from altering tests, as examined in ImpossibleBench [104]: test integrity and adequate outcome coverage are separate safeguards.

These studies narrow the proposed novelty to the case trace and the specific prospective package comparison, including an unforeseen change request. They also caution against assuming that another reviewer, a stricter check or a high hidden-test score necessarily certifies the intended deliverable. Preprints, observational studies, practitioner accounts and first-party industrial reports have different evidential roles; none substitutes for the planned controlled comparison. AgentRx's first-party release describes evidence-backed diagnosis of a trajectory's first critical failure [68]. It motivates traceable failure localisation. An independent diagnosis of the AMA mechanisms remains outstanding.

A.2.6 Optimisation with behavioural feedback

Pavlenko et al.'s *Towards an AI Software Factory for Data Systems* offers a complementary case [81]. Hotspan packages optimisation targets from telemetry, analysis and reviews; Darwin explores candidate changes through code–test–debug–reflect cycles. The setting is existing, human-architected systems with stable semantics and measurable objectives such as latency. Reported optimisation gains are evidence against treating high agent throughput as inherently unproductive, not a controlled estimate of end-to-end productivity, general maintainability or future changeability. Their application-work productivity calculation excludes infrastructure and onboarding time, and output enters engineering review. The work establishes a precedent for bounded agent-led correction; it does not isolate consolidation between feature additions or product-goal selection.

A.2.7 The practitioner conversation

Waseem et al. describe rapid generation alongside architectural inconsistency, security concerns and maintenance overhead [97]. That trade-off is not claimed as new here; the contribution is the case trace and prospective package comparison.

Beck supplies the conceptual bridge: augmented coding retains care for code, complexity and tests; alternating features with consolidation aims to restore options consumed by additions [12, 13, 16]. Understanding and abstraction are distinct from producing more artefacts [15]; learning

through working software is developed in Beck's talk [16]. These arguments motivate testing bounded reflective work, not an established remedy or a permanent human–agent boundary. Extended practitioner accounts remain in the ancillary supplement.

Spec-driven development adds structure, but not an automatic outcome oracle. Spec Kit proposes specification, planning, tasks and implementation; Böckeler describes both ignored and over-followed instructions; and a study of practice reports reliability benefits alongside planning and review costs [1, 20, 30].

A.2.8 Engineering principles behind consolidation

The engineering lineage is more specific than a general call for better code. Beck links small structural changes to test feedback [8, 11]; Gamma, Helm, Johnson and Vlissides describe reusable designs with explicit applicability, consequences and trade-offs [39]. Martin emphasises executable technical feedback [66], while Fowler's design stamina hypothesis connects attention to design with the cost of later features [36]. These ideas motivate the consolidation checklist: locate responsibilities, expose dependencies, preserve contracts and leave a concise account of design choices. They are design rationales, not empirical evidence that the proposed agent intervention is effective. Applying more named patterns is not itself an outcome; retained behaviour and the cost of subsequent change are the proposed tests.

Naur distinguishes a programmer's theory of the system from its documentary record; Brooks locates essential difficulty in the conceptual construct, not its representation [22, 74]. Royce and Gall motivate early experience of working behaviour [38, 87]. XP's short releases and tests [8, 9], Crystal Clear's incremental team practice [27], tracer bullets [49] and Agile's preference for working software [7] make that feedback concrete. Small reversible structural changes supply a consolidation practice [11].

Automation can shift effort from producing to monitoring and intervention, with substantial metacognitive demands [3, 91, 94]. Associations between confidence in AI and reduced scrutiny do not establish causation [57]. Goodhart and Strathern motivate checking completion proxies against their intended outcomes [43, 93].

A.2.9 Hypervelocity Engineering (HVE Core)

HVE Core is an open-source framework from Microsoft that supplies workflow structure for GitHub Copilot; the author's coordinator provided autonomous worker dispatch. Its Research, Plan, Implement and Review (RPI) workflow leaves dated records under `.copilot-tracking`, with explicit human stops and review. The case used those file-based handovers and its Experiment Designer's Minimum Viable Experiment method [69]; the coordinator supplied the separate autonomous delivery loop. Framework product details are secondary context, retained in the ancillary supplement. The author reports using HVE Core plugin snapshot `plugins-v3.3.106` throughout the case. Its snapshot commit is `2df5013e29ebfd2ce61bcd0cf021d8c8186482eb`, generated from source commit `e9af018fe5ffbfbc72321a8bca21fb987359af08`. Historical use is an author account; the snapshot identifiers do not establish an installation date.

A.3 Expanded case interpretations

A.3.1 Specification inflation

Interpretation: specification growth outpaced the behaviour validated through product use. By 22 October the BRD had reached version 0.19 and about 37,900 words, with requirement identifiers up to FR-052 and AC-103; the PRD had reached version 0.18 and about 33,900 words, with identifiers up to FR-061 and AC-110. The backlog inherited that scale and grew to 192 issues. Cross-cutting concerns were appended to existing issues rather than sequenced as their own work: a responsible-AI section on 15 October and an AI-consumption section on 22 October.

Issue #126, on pilot release gates, ended with seventeen acceptance criteria in three sections. Even the request for the smallest runnable preview arrived with thirteen.

This expansion began before the autonomous backlog loop, including after participant prototype and MVP feedback. The requirements agents sought fully specified BRDs and PRDs, named approvers and resolved ownership before building. The author requested explicit demonstrator assumptions and imperfect increments that could produce feedback. Smaller, iterative requirements documents are a plausible alternative, not an intervention tested in this case.

The pattern resembles Böckeler's small bug that became sixteen acceptance criteria [20], at the scale of a programme, and the risks of late integration described by Royce and Gall [38, 87]. Brooks locates essential difficulty in the conceptual construct rather than its representation [22]; repeating a requirement in more places need not resolve its ambiguity. The benchmark evidence points the same way: agents given only a requirements document lose global coherence [32]. Beck's metaphor fits: features consume future options [13, 16]. Extending that argument to acceptance criteria is the present author's interpretation, not Beck's wording. The early-warning signal is the ratio of specification growth to behaviour a human has validated in the same period. No replacement journey was reported complete during the loop, but a systematic count of validated behaviours was not collected; this ratio cannot be estimated from the document counts alone.

A.3.2 Proxy capture

In this case, artefacts made into gates appeared to displace outcomes. Three patterns motivate this interpretation.

Merged pull requests are a weak proxy for delivered value. Murphy-Hill et al. report roughly 24% more merges among adopters in Microsoft's rollout, while explicitly distinguishing merges from value [73]. Peralta et al. studied 9,799 human-reviewed agent pull requests; in their 717-PR manually inspected sample, only 35.7% of rejections reflected clear agent failures and 15.4% of merges required explicit reviewer involvement [82]. AIDev shows the scale of this measurement problem: 932,791 agent-authored pull requests from five agents across 116,211 repositories, not a count of delivered outcomes [60].

Process. When the first waves of sessions skipped the research and planning phases, the author made the presence of their artefacts a merge condition. From then on every pull request carried research, plan, change and review records; the artefacts were guaranteed, the thinking they were meant to evidence was not. Pull request #445 changed two lines of behaviour and added a 78-line evidence record that reports its own artefact completion as 100%. HVE Core treats these files as working memory. Substantial tracking output was already committed before the added merge condition; that condition formalised an existing practice rather than explaining its onset.

In the first-parent integration view, tracking-artefact additions were 10.27 times product-source additions during the loop, compared with 29.28 before it (anc/repo_history_metrics.out .txt). Artefact volume remained high during the loop, while its ratio to product-source additions fell. Churn alone does not establish code decay.

Evaluation. The rubric scorer, a language model reading the repository, rated implementation completeness at five out of five, citing a README and a check script, at a time when the author could not sign in to the replacement. The score and blocked sign-in occurred during the same period. It remains necessary to check whether the score covered that release and journey (Section 3.6). The judge read repository text rather than executing the intended journey. Neither its repeatability nor its agreement with observed product behaviour was measured; its score cannot substitute for that behavioural check.

Evidence boundaries. The author had instructed the agents never to fabricate approvals or overstate evidence. The record contains explicit statements of unmet acceptance, and lexical evidence-boundary density more than tripled. These observations do not establish exhaustive

compliance or measure honesty. Issue #349 records that a browser redirect "is not successful sign-in". The agents were right, and still nobody could sign in. Honest boundary statements became the output.

Goodhart and Strathern predict the first two levels [43, 93]. Beck joked that an agent seemed ready to delete tests and pretend it had finished [14]; ImpossibleBench measures test exploitation more generally, not a counted test-deletion category [104]. The METR study adds the human side: the feeling of progress is an unreliable instrument [17]. The early-warning signals are the ratio of evidence artefacts to behaviour changes, the share of merged pull requests that change what a user can do, boundary-phrase density, and any divergence between automated scores and a live demo.

A.3.3 Learning evaporation

Some recorded lessons appeared to fall off the loop's read path, while important design decisions survived. Maintaining the structured log and memory became difficult as the record grew. Important design decisions survived, even as some recorded lessons appeared to be lost. Before the loop, participant use of the MVP produced concrete lessons: the simplicity requirements, a deliberately public development profile for throwaway stacks, and the callback, role and staff-binding steps that made sign-in work. All were recorded, in the PRD, in a simplicity review and in infrastructure runbooks. The stated brief format carried issue text, acceptance criteria and repository conventions; actual session access to the earlier lessons is unverified. The November search finds retained Entra, registration, staff-binding and runbook references in committed research/plans, including 16 October records. This weakens a blanket disappearance claim without establishing that agents accessed or used the material. The log cites the simplicity requirements for the last time on 13 October; the development profile appears only on 8 October. The migrated deployment lacked the registration and staff bindings established on 12 October, and sign-in was blocked.

The read path grew. The knowledge-base index, which every agent is told to read first, accumulated 1,069 words of issue-specific status notes ahead of its first instruction. The index, the log, the BRD and the PRD, the documents that define the project's state and intent, total about 139,000 words. No provider-matched tokenizer or actual session-input record was available; prose token-per-word heuristics are unsuitable for these identifiers, paths and tables. Long contexts can reduce reliable retrieval even when the material fits in the context window [47, 61]; the case does not measure any session's effective attention capacity.

Instruction debt and learning evaporation can therefore coexist. A correction may remain in an expanding agent definition while the evidence explaining when it applies falls off the next session's read path. The result can be a larger instruction set with weaker access to the relevant lesson. However, unavailable guidance, accessed-but-misapplied guidance and guidance insufficient for the task are different explanations [51, Secs. 5.2–5.5]. Without actual session inputs and access/use traces, this case cannot distinguish them.

Naur explains why recording was not enough. A program's theory lives in the people who build it, and a second team with full documentation can still destroy a design's simplicity [74]. In an agent loop, every session may face an analogous reconstruction problem. The comparison with Naur is interpretive. The case provides no measure of lost understanding. Yan's advice to share context appears within an argument against multi-agent designs [99], not evidence that adding agents solves retrieval. The multi-agent failure taxonomy [23] identifies several coordination problems; it does not establish context loss as a prevalent cause in this case. HVE Core's choice to carry context in files on disk is necessary but not sufficient: the files must be on the read path, short enough to read and, ideally, executable. The early-warning signals are lessons that exist only as prose, a read path that grows faster than the code, and problems that have to be solved twice.

Pavlenko et al.'s World Model offers a different design: preserve raw evidence, derive re-

buildable views and retrieve compact, task-specific context with provenance and access controls. Updates are reviewed, and measured outcomes feed later decisions [81]. This distinguishes storing lessons from making them usable in the next decision; a small read path need not mean discarding history. This architecture offers another way to retrieve accumulated lessons. Its effect on learning evaporation in this case remains untested.

A.3.4 Process-gate inflation

Interpretation: repeated gates appeared disproportionately burdensome; comparative effort was not measured. Each new gate required execution and review. Within ten days the coordinator required, for every issue, the full RPI loop, a complete local gate before every push and after every correction or rebase, independent review, up to two correction cycles, board updates and evidence-based ticking of acceptance criteria. Hosted CI minutes had to be rationed, the full gate had to be interrupted by explicit exception for a two-line fix, and paid cloud validation stayed blocked even after the cost ceiling was raised.

The author perceived attention as more constrained than compute, but their relative scarcity was not measured. Reviewing eighteen merged pull requests a day in depth was judged impractical, and merging was delegated to the coordinator. The author still answered approvals and technical blockers, but stopped getting feedback from using a replacement product. RPI critiques and reviews addressed the current issue rather than the cumulative system; there was no regular phase for reflective consolidation or generalisation across completed issues. This is Bainbridge's irony in miniature [3]: the automation took the part of the work that supplied direct product-use feedback and left the participant monitoring a stream of plausible, well-documented changes. It matches the shift from producing to evaluating reported for generative AI [91], the delegation demands it creates [94]. Higher confidence in AI was associated with reduced self-reported critical scrutiny in the cited study [57], not established as its cause. Murphy's diagnosis is a plausible interpretation: throughput increased while the path to a usable outcome remained blocked [42, 72]. The bottleneck itself was not instrumented. Martin's argument for executable feedback from tests and code-analysis tools [66] motivates a proposed gate that checks product behaviour rather than inferring it from paperwork; the interview does not establish that such a gate would have prevented this failure. The early-warning signals are gate time per change, human review minutes per merged pull request, work in progress, and cost per merged pull request compared with cost per accepted outcome.

A.3.5 Feedback starvation

Candidate mechanism: insufficient independent, outcome-level feedback even though artefact-level validation is abundant. This differs from merely having few tests, a long feedback delay or missing context: many checks can pass while none disconfirms the claim that the intended user journey works. Outcome-level independence here means evidence obtained by exercising the outcome rather than rereading its completion record; it need not imply a different model. It does not establish independent human judgment. The loop prioritised completing the specified plan rather than shortening the path to such feedback. Feedback starvation is a central interpretive proposition, although the case cannot separate its contribution from the other mechanisms. Nothing in the coordinator kept a usable system alive: no invariant required the participant's sign-in and core workflow to remain usable after every merge. Completion was assessed through artefacts, criteria and gates. In the MVP week the feedback loop included intermittent participant use, with some changes made the same day. From 15 October the first priority was restructuring code, the working MVP was frozen, and its replacement had not become usable by the time of the case cut-off. When a usable increment was requested, the request was absorbed by the same machinery: a label, new issues, thirteen acceptance criteria and the full set of gates.

Several older practices in Section A.2.8 make feedback and working behaviour explicit: short

releases and test-first development [8, 9], incremental delivery in Crystal Clear [27], tracer bullets [49], and working software over comprehensive documentation [7]. Beck's warning that fully agentic work without feedback goes off the rails [16], DORA's emphasis on small batches and user focus [33] and Murphy's list of real bottlenecks, which includes missing feedback [72], motivate this interpretation but do not independently diagnose the case. The early-warning signals are the time from the start of a cycle to the first human use of a running increment, and the age of the last increment a human has verified as usable. The case records no completed replacement journey during the ten-day loop; the exact last-use timestamp remains unverified.

A.4 Case rival-explanation register

The main interpretation is in Section 5.2. The register preserves recorded rivals separately from untested alternatives. A limit is not grounds for ruling an explanation out. Score/deployment correspondence remains unverified (Section 3.6).

Table 13: Rival explanations with direct case records.

Explanation	Case support	Counter-evidence or limits	Treatment
Migration decision	ADR 0006 (issue #196) froze feature work in the MVP and prioritised a new layout.	Health and restart checks passed, not usability.	Migration and coordination changed together; autonomy alone is not identified.
Capstone rubric	Research and documentation are rewarded; full completeness marks coexist with blocked sign-in.	A working demonstrator is required; release/scope linkage is pending.	Possible incentive, not a measured effect.
Owner overrides and priorities	Committed RPI records, full gates and delegated merging; PR #445 required an exception.	The MVP used agents successfully; HVE Core defaults retain human stops.	Owner choices are part of the case.
Model and tool changes	Versions changed during the study.	No matched rerun under fixed versions.	Actual session configuration remains partly unknown.
CI and cloud budgets	Issue #413 moved validation locally; costs prompted a higher ceiling and reduced resources.	Merges continued without established deployed usability.	More budget is not proven to resolve delivery.
Identity and sign-in setup	Missing registration/staff bindings; issue #349 distinguishes redirect from sign-in.	Those steps worked on 12 October.	Concrete configuration obstacle, not proof of code decay or lost knowledge.

Table 14: Additional plausible rivals and unperformed discriminating checks.

Explanation	Reason considered	Counter-evidence or limits	Evidence still needed
External services and deployment drift	Model/identity configuration differs from the MVP.	No provider outage is established; health checks were narrower.	Versioned deployment/configuration and dependency-health records for the journey.
Scope expansion	RAI and consumption criteria added to issues; further resident-preview obligations.	Expansion predates autonomy; some controls are justified.	Original/added scope, sequencing and time-to-outcome comparison.
Inadequate acceptance tests	Checks passed without completed real-user sign-in; scorer read artefacts.	Bounded, honest tests need not be poor tests.	Capability-to-task-oracle mapping and execution evidence.
Rate and context limits	Parallel sessions and a large read path.	No runtime rate-limit, truncation or attention dataset analysed.	Actual inputs and blocked/retried request events.
Specification ambiguity	Selective review of long, cross-referenced requirements.	Important domain/consent decisions survived; ambiguity not independently coded.	Independent interpretation and conflicting acceptance examples.
Ordinary prototype debt	Rapid MVP followed by restructuring and rebuilding.	MVP worked intermittently; no comparable human-led migration.	Failure analysis distinguishing inherited debt from introduced faults.

A.5 Candidate-mechanism evidence audits

These are operational boundaries for this paper, not validated necessary and sufficient conditions for all agentic development. The indicators refer to Table 1 and the case chronology. Experimental implications state what evidence would discriminate an explanation; the tagged experiment does not manipulate the five candidates separately.

A.5.1 Specification inflation

Element	Audit
Definition	Requirements and acceptance obligations expand faster than independently validated behaviour, without evidence that the added scope is needed for the next usable increment. Length alone is insufficient.
Case indicators	O3: BRD/PRD growth, 192 issues and cross-cutting additions; the smallest resident preview had thirteen criteria.
Counter-evidence	Growth began before autonomy and sometimes followed the owner's feedback; regulatory and consent controls can justify detail.
Rival explanations	Genuine scope expansion, specification ambiguity, capstone incentives and an ambitious migration rather than agent-driven inflation.
Experimental implication	Compare smaller requirements horizons at matched scope and resources. The current fixed-suite experiment cannot identify this mechanism, even if consolidation helps.

A.5.2 Proxy capture

Element	Audit
Definition	A completion proxy becomes a decision criterion and remains positive when an independent test of the intended outcome is negative. Proxy use alone does not establish capture.
Case indicators	O2–O3: a 5/5 score and blocked sign-in coexist in the chronology; release/scope linkage is pending. Mandatory artefact presence and the evidence record for PR #445 are separate indicators.
Counter-evidence	Agents explicitly reported unmet acceptance; neither fabricated approval nor deliberate deception is established.
Rival explanations	An inadequate scorer or acceptance oracle, different evaluation targets, and a deployment failure invisible to repository review.
Experimental implication	Disagreement between visible completion and hidden behavioural tests would expose proxy limits. It would not by itself establish intentional test gaming or explain why a gate was chosen.

A.5.3 Learning evaporation

Element	Audit
Definition	A recorded lesson relevant to a later decision is absent from, unusable on, or not applied through that decision's effective context path, and the associated problem recurs. A missing citation alone is insufficient.
Case indicators	O4: simplicity references cease; working sign-in setup is not preserved; state and intent documents reach about 139,000 words.
Counter-evidence	Important domain decisions survived; the lesson could have been known but deferred. Actual session attention was not measured.
Rival explanations	Deliberate migration sequencing, deployment-specific configuration, context limits, or conflicting requirements rather than lost knowledge.
Experimental implication	Trace actual inputs and repeated faults. B–C cannot isolate memory quality because DESIGN and REVIEW notes and acting permissions differ. A matched-context study would be needed.

A.5.4 Process-gate inflation

Element	Audit
Definition	Additional procedural obligations increase per-change burden without demonstrated proportional improvement in the targeted outcome. More gates alone are not evidence that their net benefit is negative.
Case indicators	O5: full RPI and validation on each correction/rebase; hosted-minute rationing; explicit exception for PR #445.
Counter-evidence	Checks caught bounded defects and enforced evidence integrity. Gate duration and human review effort were not measured per PR.
Rival explanations	Legitimate safety and privacy requirements, budget constraints, new deployment complexity and owner-selected risk tolerance.
Experimental implication	Measure gate time and accepted outcomes under matched risk. The pilot's resource description concerns the package, not the effect of removing this repository's required gates.

A.5.5 Feedback starvation

Element	Audit
Definition	Independent outcome-level feedback is insufficient to correct delivery decisions despite abundant artefact-level validation. The defining gap is between the evaluated proxy and intended outcome, not test count alone.
Case indicators	O1–O2 and O5: working MVP frozen; replacement unusable; passing checks and continued approvals did not restore a completed user task.
Counter-evidence	Human oversight continued; agents surfaced blockers; engineering checks provided real, though narrower, feedback.
Rival explanations	Identity and service configuration, migration priority, incomplete acceptance coverage, prototype debt or insufficient resources.
Experimental implication	Hidden retention tests and CR1 supply behavioural feedback to the evaluator. The experiment does not test live deployment recovery, product selection or a human-use checkpoint cadence.

A.6 Measures and lexicon

Log entries. The knowledge-base log (`docs/reference/log.md`) was split into entries at each second-level heading that begins with a date. An entry belongs to the date in its heading. Word counts use whitespace tokenisation of the entry, heading included.

Evidence-boundary phrases. Each entry was searched, case-insensitively and on word boundaries, for these patterns: *unmet*; *not accepted*; *not inferred*; *is inferred*; *remain(s) open*; *remain(s) unproved*; *remain(s) unverified*; *not claimed*; *not established*; and *no* followed, within forty letters, spaces, commas, slashes or hyphens, by *is* or *was* and then *claimed* or *inferred*. Density is matches per 1,000 words.

Reference tracing. An entry cites the simplicity requirements if it contains an identifier from SA-001 to SA-010 or the word *simplicity*. It cites the development profile if it contains *deploymentProfile*, *development profile* or *allowPublicNetworkAccess*.

Repository counts. Issue and pull-request counts come from the GitHub search API on 24 October 2026 (for example, `is:pr is:merged merged:>=2026-10-15`). Document sizes are whitespace word counts of the files at commit `29c6b51`.

Proposed measures. *Time to first human use*: elapsed time from the start of a delivery cycle to the first time a person uses the running increment for its purpose. *Age of the last usable increment*: time since a person last verified that the deployed system could be used for its main journey. *Behaviour-changing merge ratio*: the share of merged pull requests that change what a user can observe in the running increment. *Cost per accepted outcome*: total model, tool and cloud cost divided by the number of outcomes a human accepted in the same period.

A.7 Consolidation step prompt

Verbatim frozen v4 prompt from execution source e8707ecc8965d3f81f5142a2603042bfca1aab15, not the amended study's approved prompt. The archived source is byte-identical, SHA-256 75f646edbe04e1e3403b092b13edd6b66615a3362e67ad49744d229cdcb94f29. Display wrapping does not alter its text. Braces mark runner parameters.

```
<!-- markdownlint-disable-file -->
# Consolidation step prompt (arm B)
```

```
Do not add or change features. Your job is to leave the code better than you found it without
↪  changing its behaviour.

1. Run `make check`, then read reports/quality.md (functions above complexity 10, duplicated
↪  blocks, lint warnings, file sizes).
2. Review the last feature's diff (`git diff feature-start feature-end`) and the hotspots
↪  against this checklist:
   * duplicated logic;
   * long or complex functions;
   * unclear names;
   * dead code;
   * responsibilities in the wrong place, or rules from one feature leaking into another;
   * tests that check implementation details instead of behaviour.
3. Make small, behaviour-preserving changes. Apply every change through `./tcr "<message>"`: it
↪  runs all tests and commits if they pass, or reverts if they fail.
4. Never modify or delete files in tests/examples/. You may add tests that pin current
↪  behaviour before you refactor.
5. Rewrite DESIGN.md in at most 300 words: the current structure, where each business rule
↪  lives, and known debts. Replace its content; do not append.
6. Stop when the quality budget is met (no function above complexity 10, duplicated lines below
↪  3%, zero warnings in the configured lint rules) or when your time budget
↪  ({consolidation_budget}) is used.
7. Finish with a summary of at most five lines: what you changed and what debt remains.
```

A.8 Deferred intervention measures

The study excludes intervention-rate outcomes but retains help and administrative records for eligibility and cost. These definitions are proposed future measures, not collected results. HG-DAgger and ThriftyDAgger supply cross-domain measurement precedents, not coding-agent effectiveness evidence [48, 53].

For run r, n_r is attempted feature cycles, including assigned post-steps and failed cycles, and h_{rj} qualifying corrective-help events. Observed human intervention rate would be

$$\mathrm{HIR}_r = \frac{1}{n_r} \sum_{j=1}^{n_r} \mathbf{1}\{h_{rj} > 0\}. \tag{3}$$

Event counts and active human minutes are separate; repeated help in one cycle counts once for incidence. With $d_{rj} = 1$ for an explicit help request or a block needing corrective human action,

$$\mathrm{IDR}_r = \frac{1}{n_r} \sum_{j=1}^{n_r} d_{rj}. \tag{4}$$

Requests and supplied help differ. A timeout alone is not a help request; zero attempts gives undefined rates. A future annotation protocol needs event/cycle/phase locators, coverage checks and adjudication; absent records cannot be zero. Routine launches, planned approvals and safety stops differ from corrective rescue. Low intervention must not reward silent failure or discourage necessary human judgement.

A.9 Historical freeze provenance

The historical v4 execution source was e8707ecc8965d3f81f5142a2603042bfca1aab15, with manifest-only commit 03c50856838b539780d57216af2a4cf6744187d7. Its attached osf-v4-2026-10-31.tar.gz checksum was 1a42fac40a4f1aa82df59c8c3f7f8d6dce37d22502c5952c398362ac63d6bd65. The runtime image was sha256:090220b0b172ec205f1ea473d7c3a223eec5d7d619eb21acdcf15c9289287fa3. These identify historical inputs, not a freeze or approval of the five-arm amendment. The original registration at https://osf.io/ywsrv/ is owner-reported approved after submission on 28 October; the inspected v4 prospective amendment flag was false and

its completion date null. New approval and actual prospective recording cannot be inferred or backdated.